Long-Horizon LLM Agents: Horizons, Models, Harness Design, and Evaluation

Anonymous ACL submission

Abstract

LLM agents are increasingly expected to solve tasks that require extended interaction with tools, software environments, web interfaces, databases, and other mutable external states. Long-horizon LLM agents mark a shift from isolated model responses to sustained, stateful, and goal-directed execution. They must plan across multiple stages, preserve task context, coordinate tool use, verify intermediate outcomes, and recover from failures as errors accumulate over dependent trajectories. This survey provides a systematic analysis of long-horizon LLM agents through four perspectives: horizon taxonomy, model capability, harness design, and evaluation methodology. We introduce a reliable execution horizon as a unifying lens for organizing existing work, exposing key limitations, and motivating future research on reliable long-horizon agency.

1 Introduction

Large language model (LLM) agents are transforming NLP systems from isolated prediction engines into systems for delegated execution. Rather than producing a single response to a static input, agents are increasingly expected to browse websites (Zhou et al., 2024b), edit code repositories (Jimenez et al., 2024), operate graphical interfaces (Xie et al., 2024b), call APIs and query databases (Trivedi et al., 2024a), and coordinate tool-agent-user workflows (**?**). In these settings, the basic unit of behavior is no longer an individual model output, but an interactive trajectory in which actions alter external state and thereby condition subsequent observations, decisions, and success criteria. This transition marks a shift from static language understanding toward sustained, stateful, and goal-directed execution.

[**TODO/TBD:** Add a figure about evolution of long horizon llm agents]

Despite rapid progress, the notion of *long-horizon* agency remains under-specified. Existing work often associates horizon with long contexts, many interaction turns, prolonged wall-clock duration, large numbers of tool calls, or difficult benchmark instances. These proxies are useful but insufficient: an agent may process a long document without performing long-horizon execution, while a comparatively short interaction may become long-horizon when early actions create constraints that later actions must respect (Sinha et al., 2025; Wang et al., 2026c). We therefore define horizon as the extent over which an agent must preserve goal-relevant information, maintain a valid representation of the world state, select dependent actions, and verify delayed consequences. This view separates horizon from surface length and instead emphasizes dependency depth, mutable state, memory persistence, grounding, delayed verification, and repeated-run reliability. METR-style long-task evaluation captures one dimension of horizon through the human time required for tasks that systems can complete at a specified success rate (Kwa et al., 2026), while τ-bench captures another by evaluating whether tool-using agents can succeed consistently across repeated trials rather than only once (**?**).

In this survey, we argue that long-horizon LLM agents should be studied as coupled *model–harness–environment–evaluation* systems. Models determine local transition quality by decomposing goals and reasoning over intermediate steps (Wei et al., 2022), grounding actions through tool-use learning (Schick et al., 2023), revising after feedback (Shinn et al., 2023), and learning from delayed rewards in agentic settings (Li et al., 2025a). However, model capability alone does not determine the effective horizon of an agent. The surrounding harness—including memory management (Packer et al., 2023), programmatic prompt and module composition (Khattab et al., 2024), software-agent execution environments (Wang et al., 2025e), and mixed-initiative

human oversight (Mozannar et al., 2025)—shapes whether local errors are prevented, detected, isolated, or allowed to propagate. Evaluation protocols further determine whether observed improvements reflect genuine reliable autonomy or artifacts of sampling budget, scaffolding, benchmark contamination, or unstable environments (Guo et al., 2024; Kapoor et al., 2025).

We organize existing work through the lens of the *reliable execution horizon*: the range over which an agent can continue acting while remaining correct, grounded, recoverable, and cost-effective. This construct explains why long-horizon settings are qualitatively different from single-step prediction: small local weaknesses can compound when errors enter memory, corrupt external state, invalidate future preconditions, or mislead subsequent verification (Ord, 2025; Khanal et al., 2026). Accordingly, this survey analyzes long-horizon LLM agents from four perspectives aligned with the title. Under *Horizons*, we formulate what makes an agent task long-horizon and introduce measurement axes for horizon stress. Under *Models*, we synthesize model-level determinants that reduce local transition error. Under *Harness Design*, we examine system scaffolding that stabilizes execution through memory, tools, verification, rollback, and oversight. Under *Evaluation*, we review benchmarks such as AgentBench, LiveBench, WebArena, OSWorld, SWE-bench, AppWorld, and τ-bench (Liu et al., 2024; White et al., 2024; Zhou et al., 2024b; Xie et al., 2024b; Jimenez et al., 2024; Trivedi et al., 2024a; **?**), together with metrics and protocols for reliability, attribution, cost, contamination, and reproducibility (Ma et al., 2024; Shavit et al., 2023). We conclude that progress in long-horizon agents requires measuring and extending reliable execution horizon as a property of the full model–harness–environment–evaluation stack.

2 Taxonomy: What Are Long-Horizon Tasks?

[**TODO/Xiaozhong:** Coordinate the taxonomy section.]

2.1 Working Definition: When Does Execution Become Long-Horizon?

A task becomes long-horizon for an LLM agent when success depends on a trajectory whose later decisions are conditioned on earlier actions, mutable state, external feedback, and verification outcomes. Horizon is therefore not identical to prompt length, wall-clock runtime, or the number of visible turns. A task is long-horizon when the execution process itself becomes the object being tested: the agent must preserve task-relevant state, act through dependent transitions, update its beliefs from tools or environments, and avoid letting local errors become future context. Long-horizon execution is therefore a reliability problem over a stateful trajectory, rather than a larger version of single-turn prediction.

The boundary can be stated as a compression test. If a task can be decomposed into independent one-step predictions without changing the success condition, then it is multi-step but not strongly long-horizon. If the same decomposition destroys the task because an early action changes later feasible actions, because a tool result must be remembered, because a database or file state has changed, or because the final answer must be checked against external state, then the task has entered the long-horizon regime. This explains why long-horizon tasks may have few visible actions but deep dependency, while a long list of independent actions may remain shallow.

Recent benchmarks motivate this distinction. MINT shows that tool use and natural-language feedback change evaluation from single-turn answering to multi-turn interaction (Wang et al., 2024c). AgentBoard argues that final success rate alone hides intermediate progress and failure locations in partially observable multi-round environments (Ma et al., 2024). AppWorld makes the statefulness explicit by evaluating agents in a controllable world of apps with state-based unit tests that also check unintended side effects (Trivedi et al., 2024a). Tau-bench adds a complementary reliability perspective: conversational agents must interact with users, APIs, and domain policies over long horizons, and the proposed pass^k metric measures whether all repeated trials succeed, not merely whether one sample succeeds (**?**).

Three layers follow from the compression test. The first is structural horizon: the depth and coupling of dependent decisions. Sinha et al. isolate execution capability by giving models the required knowledge and plan, yet still find that per-step accuracy degrades as execution length grows (Sinha et al., 2025). This indicates that long-horizon failure is not only a planning failure. It can arise from carrying out a known plan over many state transitions. HORIZON reaches a related diag-

nostic conclusion from another direction, treating long-horizon breakdown as a trajectory-level phenomenon that requires failure attribution across domains rather than a single end score (Wang et al., 2026c).

The second layer is state horizon: the duration over which information remains relevant and must be grounded correctly. State may consist of user constraints, tool outputs, retrieved evidence, files, code edits, browser observations, database rows, or hidden assumptions created by the agent itself. METR's long software task study uses a human-grounded 50% task-completion time horizon and reports that agents still struggle in messier environments, especially where feedback loops and cheap verification are weak (Kwa et al., 2026). This evidence suggests that horizon is not only about how long a task takes. It is also about how long the agent must maintain a correct representation of a changing problem.

The third layer is reliability horizon: the distance over which success remains repeatable. A pass@1 score asks whether one sampled trajectory reaches the goal. A long-horizon reliability measure asks whether success persists across repeated attempts, perturbations, and longer task durations. Rabanser et al. decompose agent reliability into consistency, robustness, predictability, and safety, showing that capability gains do not necessarily yield comparable reliability gains (Rabanser et al., 2026). Beyond pass@1 formalizes the same gap through duration-sensitive metrics such as the Reliability Decay Curve and reports that capability and reliability rankings can diverge as horizons lengthen (Khanal et al., 2026). The operational implication is direct: an agent that sometimes solves a long task is not yet reliable at that horizon.

The resulting construct separates long-horizon execution from its observable proxies. Number of turns, context length, human completion time, grounding load, dependency depth, and repeated-trial variance are useful measurement axes, but none is sufficient as a definition on its own. The underlying object is stateful trajectory reliability under dependency, feedback, and verification.

Two boundary conditions follow. Long context is not the same as long horizon: a model may receive a long input without maintaining a changing execution state, while a short-context task may still require careful state transitions and recovery. Tool use is also not sufficient by itself. A single correct API call remains a tool-use task, whereas a sequence of policy-constrained calls that changes database state and must satisfy a final invariant becomes a long-horizon agent task. The transition occurs when the agent's own actions create the conditions under which later reasoning must operate.

The same axis organizes the broader problem space. Benchmarks differ in whether they stress structural dependency, state grounding, or reliability under repetition. Model-level methods mainly reduce transition error through reasoning, search, tool-use learning, and agentic reinforcement learning. Harness-level methods mainly extend the reliable horizon through memory management, interfaces, sandboxes, feedback loops, verifiers, rollback, and trajectory monitoring. Reliable horizon is therefore a systems property. It emerges from the model, the harness, the environment, and the evaluation protocol together.

2.2 Measurement Axes: How Is Horizon Operationalized?

Because horizon is not a single scalar property, existing benchmarks operationalize it along several complementary axes. We group these operationalizations into six measurement axes: human-effort time, interaction length, context and memory demand, environment grounding, planning dependency, and evaluation reliability. These axes are not mutually exclusive categories; rather, they describe different sources of horizon stress that may co-occur in the same task.

Time / Human-Effort Horizon. The time or human-effort horizon measures task length by the amount of time a qualified human requires to complete the task. METR-style time horizons and long software task evaluations define capabilities in terms of the human completion time that an AI system can match at a given success threshold (Kwa et al., 2026). This axis is attractive because it provides an interpretable unit of task length and supports comparison across heterogeneous task suites.

This axis is especially useful for software engineering, AI R&D, and algorithm engineering. RE-Bench compares agents with expert humans in multi-hour ML research environments (Wijk et al., 2024), while ALE-Bench evaluates long-horizon objective-driven algorithm engineering through repeated submission, scoring, visualization, and refinement (Imajuku et al., 2026). In these settings, the horizon is grounded in human labor time even when the visible number of agent actions is not the

Table 1: Three-layer working definition of long-horizon execution

Layer	What makes the horizon long	Evidence signal
Structural horizon	Later actions depend on earlier decisions, intermediate artifacts, or action ordering.	Performance changes when a plan must be executed over many dependent steps rather than solved as isolated subtasks.
State horizon	Task-relevant information must remain correct across tool calls, memory, files, databases, user feedback, or environment changes.	Evaluation must inspect trajectory state, database state, unit tests, or side effects rather than only final text.
Reliability horizon	Success must persist across repeated invocations, task durations, perturbations, and recovery opportunities.	Reliability metrics such as pass^k, reliability decay, variance, and failure attribution reveal behavior hidden by pass@1.

only source of difficulty.

Interaction / Step Horizon. The interaction or step horizon measures the number and dependency structure of actions, tool calls, GUI operations, or environment transitions. MINT shows that multi-turn tool use and language feedback create capability demands that are not visible in single-turn evaluation (Wang et al., 2024c). AgentBoard further complements binary success rate with progress rate, making it possible to measure how much of the task an agent has completed before final success or failure (Ma et al., 2024).

Ultra-long benchmarks push this axis further. UltraHorizon argues that many existing agent benchmarks remain short, often involving only a few thousand tokens and fewer than dozens of tool calls per trajectory, and instead constructs partially observable exploration tasks with trajectories reaching hundreds of thousands of tokens and hundreds of tool calls (Luo et al., 2025). Such tasks stress not only interaction count, but also hypothesis maintenance, systematic exploration, and recovery from early misleading patterns.

Context / Memory Horizon. The context or memory horizon captures whether an agent can preserve task-relevant state across long trajectories, including user constraints, intermediate artifacts, tool outputs, file edits, database state, and prior observations. Unlike ordinary long-context tasks, agent trajectories contain machine-generated representations such as tables, JSON objects, code, terminal logs, webpages, and GUI observations, and these representations are causally grounded in previous actions.

AMA-Bench makes this distinction explicit by evaluating memory in agentic applications rather than dialogue-only settings (Zhao et al., 2026b). AppWorld (Trivedi et al., 2024a), Odyssey-Bench (Wang et al., 2025c), and workplace-style benchmarks such as TheAgentCompany (Xu et al., 2026a) similarly require agents to maintain persistent cross-application state. The key challenge is not merely fitting history into a context window, but selecting actionable information, updating stale state, and reconstructing the current task state when needed.

Environment / Grounding Horizon. The environment or grounding horizon measures an agent's ability to maintain correct perception-action alignment in changing environments. WebArena introduces self-hosted realistic websites and evaluates functional correctness rather than surface-form action matching (Zhou et al., 2024b). OS-World extends this idea to real computer environments, where agents must operate across GUIs, CLIs, file systems, and multi-application workflows (Xie et al., 2024b).

This axis becomes long-horizon because every action changes the future observation stream. A wrong click, an incorrect file edit, or an unintended window switch can corrupt the environment state and mislead subsequent decisions. As a result, grounding horizon should not be understood as single-step perception accuracy, but as sustained alignment between high-level intent, observed state, and executable low-level actions.

Planning / Dependency Horizon. The planning or dependency horizon measures the depth of global constraints, temporal dependencies, resource limits, and delayed consequences. TravelPlanner is a representative example: agents must retrieve information from large travel databases and synthesize plans satisfying budget, route, lodging, cuisine and commonsense constraints (Xie et al., 2024a). Even when relevant information is provided, satisfying all constraints remains difficult, indicating that the bottleneck is not merely tool use but global dependency management.

The defining failure mode in this axis is that local progress does not guarantee global coherence. An agent may complete plausible subtasks while violating budget constraints, temporal consistency, route feasibility, test suites, or software specifications. Constraint-heavy planning benchmarks such as Robotouille (Gonzalez-Pumariega et al., 2025) and software repair benchmarks such as SWE-bench (Jimenez et al., 2024) expose this dependency horizon because early decisions restrict later feasible solutions and local fixes can introduce new global failures.

Evaluation / Reliability Horizon. The evaluation or reliability horizon asks whether success is repeatable, robust, predictable, safe, and verifiable rather than merely achieved once. A science of agent reliability decomposes evaluation beyond raw task success into consistency, robustness, predictability, and safety (Rabanser et al., 2026). These dimensions become more important over long trajectories because run-to-run variance, prompt perturbations, tool faults, and error severity can all compound with task length.

Benchmarks increasingly instantiate this axis through richer evaluation protocols. τ^2-Bench evaluates conversational agents in dual-control settings where the agent must guide user-side actions as well as use its own tools (Barres et al., 2025). AppWorld and OSWorld rely on state-based or execution-based verification to judge final outcomes rather than reference trajectories (Trivedi et al., 2024a; Xie et al., 2024b). These protocols shift long-horizon evaluation from asking whether an agent can occasionally succeed to whether it can be delegated reliably.

Realistic tasks are often long along multiple axes at once. OSWorld combines interaction length, grounding difficulty, context persistence, and execution-based verification. TravelPlanner is primarily long along the planning and dependency axis, but also requires tool interaction and constraint verification. Long software tasks use human-effort time as a unifying measurement, yet concrete failures may arise from memory, testing feedback, tool use, and cross-file dependency reasoning.

We therefore view the taxonomy as a coordinate system rather than a partition of benchmarks. A benchmark can stress several axes simultaneously, and an agent may fail only under particular combinations of axes. This view provides the organizing principle for the rest of the survey: benchmark landscapes can be analyzed by which horizon stresses they instantiate, model-level methods can be understood as reducing local error rates, and harness-level methods can be understood as extending the reliable horizon through memory, feedback, verification, and recovery.

3 Benchmark Landscape: Where Long-Horizon Execution Occurs in the Real World

[**TODO/boyuan:** Coordinate the benchmark landscape section.]

[**TODO/chaofan:** for benchmark and taxonomy]

3.1 Metric Stack and Experimental Protocol

Language-agent evaluation should be organized as a metric stack rather than a single success score. Unlike static language-model benchmarks, agent benchmarks involve stochastic rollouts, tool calls, state transitions, intermediate actions, environment feedback, and resource constraints. Recent surveys therefore frame agent evaluation as a multi-dimensional problem covering capabilities, behavior, reliability, safety, interaction modes, metric computation, and evaluation infrastructure (Yehudai et al., 2026; Mohammadi et al., 2025). The goal is not merely to ask whether an agent solves a task, but to characterize how consistently it solves it, how much progress it makes before failure, whether its trajectory is valid, what state changes it induces, and at what computational or monetary cost.

This distinction is especially important for repeated evaluation. Let $s_{ij} \in \{0, 1\}$ denote whether rollout j succeeds on task i. When exactly k independent rollouts are executed per task, two useful

empirical quantities are:

$$\widehat{\text{pass@}k} = \frac{1}{n}\sum_{i=1}^{n}\mathbb{I}\left[\max_{1\le j\le k} s_{ij} = 1\right], \quad (1)$$

$$\widehat{\text{pass}^k} = \frac{1}{n}\sum_{i=1}^{n}\prod_{j=1}^{k} s_{ij}. \quad (2)$$

Here, $\text{pass@}k$ measures whether at least one attempt succeeds, whereas pass^k measures whether all k repeated attempts succeed. τ-bench makes this distinction explicit in tool-agent-user interaction, showing that one-shot success can substantially overstate behavioral consistency across trials (**?**). Beyond pass@1 further argues that reliability should be modeled as a function of task horizon, since longer tasks expose decay, variance amplification, graceful degradation, and meltdown behavior that are invisible to aggregate pass@1 (Khanal et al., 2026).

Outcome metrics should be grounded in externally checkable task states whenever possible. Benchmarks such as AgentBench (Liu et al., 2024), WebArena (Zhou et al., 2024b), OSWorld (Xie et al., 2024b), and SWE-bench (Jimenez et al., 2024) move evaluation beyond surface-form answers by tying success to interactive environments, final goal states, operating-system actions, or executable tests. This shift is essential for agents because a fluent final response may hide invalid tool calls, skipped constraints, irreversible side effects, or accidental success. Final correctness should therefore be paired with progress metrics, trajectory metrics, execution-safety metrics, latency, and cost.

The experimental unit should be an episode. Each episode should record the task instance, initial environment state, model version, scaffold configuration, tool schema, decoding parameters, random seed, full action–observation trajectory, final state, grader output, and resource vector. For each agent–benchmark pair, evaluations should include multiple rollouts per task, stratification by domain and horizon length, confidence intervals over tasks and seeds, and paired comparisons when systems are evaluated on the same task set. This protocol prevents common confounds: a higher score may reflect a stronger model, but it may also reflect more search, longer contexts, extra retries, additional verifier calls, or a more favorable harness.

Process-level evaluation is also necessary because final outcomes often underdetermine the underlying failure mode. Agent-as-a-Judge illustrates one direction: agentic evaluators can inspect intermediate trajectories and provide feedback over the task-solving process rather than judging only the final output (Zhuge et al., 2025). Such process evaluation is useful for diagnosing tool misuse, looping behavior, invalid intermediate assumptions, and partial progress. However, judge-based process scores should complement rather than replace executable validation when deterministic graders are available.

Finally, evaluation infrastructure should make resource and implementation effects explicit. Holistic leaderboards such as HAL emphasize standardized harnesses, shared logs, large-scale rollouts, and cost-aware comparison (Kapoor et al., 2025). This direction is critical for agent research: without trajectory logs, budget reporting, and harness disclosure, it is difficult to determine whether progress comes from stronger models, better scaffolds, more computation, or evaluation artifacts. A mature metric stack should therefore make agent performance attributable, reproducible, and deployment-relevant.

3.2 Web Navigation and Browsing

Web navigation and browsing require agents to work in open and changing web environments. Agents must not only click, type, and search, but also track page states, check evidence, and avoid unsafe actions across long trajectories.

Interface Execution. Interface execution focuses on whether agents can turn user instructions into correct webpage operations. The main failure mode is action grounding: the agent may understand the goal but click the wrong element, fill the wrong field, or choose an invalid action. Some benchmarks use controlled or simulated websites to test step-by-step UI operation (Shi et al., 2017; Yao et al., 2022). Others extend this setting to real websites, multi-domain tasks, dialogue-based instructions, and desktop–web interfaces, where page layouts and user goals are more diverse (Deng et al., 2023; Lù et al.; Xu et al.; Wang et al.; Kapoor et al., 2024).

Trajectory Robustness. Trajectory robustness focuses on whether agents can maintain a correct path over many web interactions. Here, the key problem is not a single action, but state drift: an early mistake may move the agent to the wrong page state, and later actions may look reasonable while the whole trajectory has already gone off track. Some benchmarks build functional websites

Metric layer	Typical metrics	Core question	Reporting principle
Outcome	Success rate; exact match; goal-state match; pass@1	Did the agent complete the task?	Report task-level success with clear grading rules, preferably based on executable tests or final environment states.
Repeated attempts	pass@k; best-of-k success	Can the agent succeed if given multiple tries?	Report k, sampling settings, and the selection rule; distinguish best-of-k capability from single-run deployment performance.
Reliability	pass^k; trial consistency; success variance	Does the agent succeed consistently across repeated runs?	Use multiple rollouts per task and report variance or confidence intervals, since one-shot success can hide instability (**?**Khanal et al., 2026).
Progress	Subgoal completion; milestone accuracy; partial credit	How far does the agent get before failure?	Measure intermediate progress when final success is sparse, and distinguish early collapse from near-complete execution.
Trajectory	Action validity; tool-use correctness; loop rate; process score	Was the reasoning–action path valid and efficient?	Log full trajectories and evaluate process quality in addition to final outcomes (Zhuge et al., 2025).
Execution safety	State-diff correctness; unintended actions; rollback success	Did the agent modify the external environment correctly?	Record initial and final states, specify allowed changes, and separately evaluate irreversible or harmful side effects.
Efficiency and cost	Tokens; tool calls; latency; verifier calls; monetary cost	How much resource was consumed?	Compare agents under matched budgets and report cost-normalized success or accuracy–cost trade-offs (Kapoor et al., 2025).

Table 2: A compact metric stack for language-agent evaluation. The layers separate task completion, repeated-attempt capability, reliability, progress, trajectory quality, execution safety, and resource cost.

or online environments to test agents under changing states (Zhou et al., 2024b; Pan et al.; Xue et al., 2025; Sun and Chen, 2025). Another line adds visual grounding, where screenshots, icons, layouts, and visual cues are needed to decide the next action (Koh et al., 2024; He et al., 2024; Zheng et al., 2024a; Thomas et al., 2025). These benchmarks show why long-horizon web tasks cannot be judged only by isolated action accuracy.

Trustworthy Browsing. Trustworthy browsing focuses on whether agents can produce reliable and safe outcomes in open web environments. The main failure mode shifts from wrong actions to wrong evidence, unsafe behavior, or misleading success. Some benchmarks require agents to search across webpages, compare sources, check claims, and generate citation-supported answers or reports (Nakano et al., 2022; Yoran et al., 2024; Wei et al., 2025; Zhou et al., 2025a; Gou et al., 2025; Du et al., 2025). Others add safety, privacy, and security requirements, where agents must avoid adversarial webpages, privacy leakage, unsafe operations, fabricated evidence, and shortcut behavior (Anupam et al., 2025; Levy et al., 2024; Tur et al., 2025; Ying et al., 2026; Ramesh et al., 2026; Li et al., 2026b).

Overall, web navigation benchmarks shift the focus from task completion to trajectory reliability. The central challenge is whether agents can maintain coherent, verifiable, and safe behavior across long interactions in open web environments.

3.3 Computer Use, Mobile, and Embodied Worlds

Computer-use, mobile, and embodied benchmarks become long-horizon when agents must keep what they see, what they do, and what the environment has become aligned across many dependent actions. The key failure mode is not only a wrong final answer, but gradual observation–action–state drift: a stale screen interpretation, missed object state, wrong tap, or failed manipulation can redirect the future trajectory. This distinguishes the setting from browser-only navigation, software-repository repair, and API/workplace workflows (Xie et al., 2024b; Rawles et al., 2025; Shridhar et al., 2020).

Desktop computer use. Desktop benchmarks shift evaluation from page-level navigation to full

operating-system state. OSWorld serves as the anchor benchmark, with agents operating in real computer environments and interpreting screenshots, applications, windows, files, settings, and cross-application effects (Xie et al., 2024b). Windows Agent Arena adds Windows-specific realism and scalable multimodal evaluation, while OmniACT broadens visual-action grounding across heterogeneous desktop and web interfaces (Bonatti et al., 2025; Kapoor et al., 2024). Their common challenge is tracking visual feedback, file state, and application state as they evolve within a single desktop trajectory.

Mobile interaction. Mobile benchmarks add compact screens, app-specific navigation, dynamic task parameters, and device variation. AndroidWorld evaluates agents in a dynamic Android environment, where success requires maintaining app state and acting through real mobile interfaces rather than predicting a static answer (Rawles et al., 2025). B-MoCA adds the robustness angle by testing mobile device-control agents across diverse configurations, making layout, language, and settings variation part of the challenge (Lee et al., 2024). Longer trajectories require repeated UI interpretation under shifting device state, because each tap may expose a new page or alter task-relevant settings.

Embodied worlds. Embodied benchmarks make state change physical or simulated, so moving the wrong object, missing a precondition, or failing to recover can alter the future environment. ALFRED pairs household instructions with egocentric visual observations and low-level actions, requiring agents to connect language, perception, and action across everyday tasks (Shridhar et al., 2020). BEHAVIOR-1K expands scale and realism through 1,000 simulated everyday activities, while RoboCerebra moves toward long-horizon robotic manipulation with multi-step reasoning, subtask execution, and dynamic scenes (Li et al., 2023; Han et al., 2026). Planning depth matters, but long-horizon grounding also requires sustained control over a changing world, because early mistakes may remove preconditions for later actions.

Hybrid interfaces and boundaries. Hybrid benchmarks show that computer-use trajectories increasingly cross interface boundaries. WeaveBench combines GUI, CLI, code, and file-system operations, so evaluation must judge evidence across screenshots, commands, files, and intermediate artifacts rather than endpoint state alone (Li et al., 2026b). Its mixed interface places it near software and terminal evaluation, but the central difficulty remains cross-interface grounding rather than executable software-engineering correctness. HeroBench illustrates the boundary from the planning side. Virtual worlds may contain long action sequences and structured reasoning, but when resource dependencies, numerical feasibility, and strategic planning dominate, they fit better with constraint-heavy planning than computer-use grounding (Anokhin et al., 2025). These tasks therefore test whether agents can track visual evidence, executed actions, and environment state as interfaces change over time.

3.4 Software Engineering and Terminal Work

Software-engineering and terminal benchmarks become long-horizon when coding or command-line work is no longer a one-shot generation problem but a stateful process in an executable environment. Agents must maintain a model of workspace state, coordinate edits across files, use terminal, build, test, and CI feedback, and avoid regressions. Executable work environments expose these demands directly, as repository-level issue repair and terminal-based task environments make state tracking, feedback use, and validation part of the benchmarked trajectory (Jimenez et al., 2024; Merrill et al., 2026).

Scaling repository repair. SWE-bench established repository-level issue repair as the canonical protocol, and its Verified subset further tightened task quality and evaluation reliability (Jimenez et al., 2024; OpenAI, 2024). Subsequent variants show that this protocol is still evolving. SWE-Cycle makes the hidden lifecycle explicit, including environment reconstruction and verification-test generation (Guan et al., 2026). SWE-bench-Live, SWE-MERA, and SWE-rebench V2 address staleness, contamination, language coverage, and data-scale pressures (Zhang et al., 2026a; Pavel et al., 2025; Badertdinov et al., 2026). The same line then escalates through SWE-Bench Pro's harder professional tasks, DeepSWE's original long-horizon tasks with program-based verifiers, and SWE-Marathon's ultra-long executable environments with multi-layer verification (Deng et al., 2025; Huang and Jiang, 2026; Desai et al., 2026).

Beyond issue repair, evaluation includes tasks that require agents to understand, extend, and construct software systems. FeatureBench targets complex feature implementation, while SWE-Explore

and ReCUBE isolate two pre-editing bottlenecks: localizing the relevant code and using repository context correctly (Zhou et al., 2026b; Zhang et al., 2026c; Hong et al., 2026). SWE Atlas adds codebase question answering, test writing, and refactoring. ProgramBench, NL2Repo-Bench, and RepoGenesis evaluate whole-program or whole-repository construction (Raghavendra et al., 2026; Yang et al., 2026b; Ding et al., 2026; Peng et al., 2026).

Maintained correctness over time. Long-horizon software work also tests whether changes remain correct as repositories evolve. A patch can pass immediate tests yet fail under future requirements, version transitions, CI changes, or test-suite evolution. SWE-EVO therefore evaluates software evolution from release notes and mature project histories, requiring coordinated multi-file modifications while preserving existing behavior (Le et al., 2026). SWE-CI shifts the emphasis to maintainability under CI-style repository evolution, while RoadmapBench and SWE-Chain formulate long-horizon upgrades as version or release chains in which earlier changes become the starting state for later ones (Chen et al., 2026; Xu et al., 2026e; Lam et al., 2026). TEBench complements this line by treating the maintained artifact as not only production code, but also the tests that specify and protect behavior (Shang et al., 2026).

Terminal and CI as validation. Repository success depends on the machinery that proves code works. Terminal-Bench evaluates agents in real command-line environments, and LongCLI-Bench narrows this setting to programming tasks performed through the CLI (Merrill et al., 2026; Feng et al., 2026b). BuildBench, DI-BENCH, and CI-Repair-Bench show that executable validation often begins with environment management rather than code generation, requiring agents to infer dependencies, diagnose build logs, repair configuration or workflow failures, and rerun the system under build, dependency, and CI constraints (Zhang et al., 2025b,a; Muna et al., 2026). CLI-Tool-Bench and SWE-InfraBench extend CLI-based evaluation to command-line tool generation and infrastructure-as-code edits (Hu et al., 2026; Tarasova et al., 2026). These tasks connect code changes to executable evidence, making setup, failure diagnosis, and validation part of the evaluated trajectory.

Frontier engineering and AI-R&D automation. Recent repository and terminal benchmarks extend the evaluation target from issue repair and code maintenance to longer engineering tasks. FrontierSWE-style challenge suites emphasize multi-hour tasks in performance engineering, computational science, and machine-learning systems (Chu et al., 2026). PostTrainBench evaluates agents that combine code, search, data curation, training, and evaluation loops to improve a base model under a fixed compute budget (Rank et al., 2026). These settings still rely on executable artifacts, command-line feedback, and verification, but the evaluated outcomes are open-ended engineering results and AI-R&D pipelines rather than ordinary patch production.

The resulting benchmark family treats long-horizon SWE evaluation as a problem of sustained control over changing repositories, executable environments, and imperfect verification processes, not merely as harder coding.

3.5 Tool, API, App, and Workplace Workflows

Tool- and application-use benchmarks test whether agents can remain reliable across many dependent actions. The core difficulty is not merely selecting the right tool, but preserving state, carrying information across steps, and avoiding unintended changes as the workflow unfolds.

From function calls to stateful interaction. Early benchmarks focused on tool selection and argument generation. ToolLLM scales evaluation to 16,464 RapidAPI endpoints, while BFCL measures function-call correctness through structured matching and execution-based checks (Qin et al., 2024; Patil et al., 2025). These benchmarks establish basic tool-use competence, but short-horizon call accuracy does not directly predict performance on longer workflows, where early mistakes propagate to later steps.

AppWorld shifts the evaluation target from individual calls to persistent application state. Agents interact with 457 APIs across nine applications, and state-based unit tests verify both task completion and *collateral damage* to unrelated data (Trivedi et al., 2024b). This captures a central requirement of long-horizon tool use: reaching the intended state without corrupting the surrounding environment.

From single applications to extended workflows. WorkArena evaluates enterprise tasks on the ServiceNow platform, introducing realistic interfaces and structured business processes (Drouin et al., 2024). OfficeBench broadens the action

space to workflows spanning Word, Excel, PDF, email, calendar, and shell tools (Wang et al., 2024d). Such tasks require agents to transfer information across applications and maintain consistency over multiple state transitions.

OdysseyBench adds long-range conversational dependencies: agents must recover relevant information from extended interaction histories and use it to complete multi-application tasks (Wang et al., 2025d). Its mixture of 300 transformed workflows and 302 automatically generated tasks also illustrates a recurring realism–scale trade-off: synthetic construction increases coverage, but requires careful validation of task quality and representativeness.

Toward integrated workplaces. TheAgentCompany combines browsing, coding, enterprise services, and communication with simulated coworkers in a single organizational environment (Xu et al., 2026b). Its checkpoint-based evaluation distinguishes partial progress from full completion; even the strongest reported agents complete only about 30% of tasks end to end. This gap suggests that current systems can often solve isolated subtasks but struggle to maintain coherence across an entire workflow.

Overall, these benchmarks expose three related dimensions of difficulty: *compositional depth*, *persistent state*, and *environmental breadth*. Evaluation has accordingly moved from call-level matching toward executable checks, state assertions, and checkpointed progress. However, current metrics remain weak at identifying process-level failures, such as stale-state reasoning, failed recovery, unnecessary side effects, and procedural violations. Developing evaluators that localize these failures without penalizing valid alternative trajectories remains an important open challenge.

3.6 Constraint-Heavy Planning and Scientific Work

Where the web, computer-use, and software settings mostly stress getting each interface action right, the benchmarks in this group stress keeping a plan globally consistent as constraints, dependencies, and intermediate results accumulate. A step that looks locally reasonable can still leave the final plan infeasible. Many of these benchmarks also involve heavy tool use and database queries, but we group them by their dominant source of difficulty, global constraint satisfaction.

Real-world and natural-language planning. TravelPlanner (Xie et al., 2024a) asks an agent to assemble a multi-day itinerary from a database of roughly four million records, satisfying explicit budget and preference constraints together with implicit commonsense ones; across 1,225 queries even GPT-4 returns a fully valid plan in just 0.6% of cases, and ReAct- or Reflexion-style strategies break down once several constraints interact. NATURAL PLAN (Zheng et al., 2024b) asks whether the bottleneck is tool use or planning itself: it feeds tool outputs such as Google Flights, Maps, and Calendar directly into the context, removing the tool-use step, yet accuracy still collapses with scale, falling below 5% once ten cities are involved, and self-correction does not help. DeepPlanning (Zhang et al., 2026d) keeps a similar scenario but tightens the evaluation, grading minute-level travel and shopping plans with rule-based checkers in offline sandboxes and separating proactive information gathering from local and global constraint reasoning. Its error analysis is especially detailed, showing that frontier agents skip required tool calls on long horizons and rarely check global consistency or backtrack.

Difficulty built into a simulated world. Because the difficulty of the real-world suites is inherited from the domain, no single source of hardness can be isolated or varied on its own: TravelPlanner can only separate planning from tool use by adding a second, sole-planning mode, and NATURAL PLAN has to remove tools from the task altogether. The benchmarks in this group instead construct the difficulty themselves, introducing a chosen kind of structure that can be controlled directly. MinePlanner (Hill et al., 2023) formalizes 45 Minecraft building and navigation tasks in PDDL, the standard language for classical planning, where the hardest instances contain thousands of objects, most of them irrelevant to the goal. Its difficulty is not specific to LLMs: even dedicated classical planners run out of memory or hit the time limit on these instances before returning a plan. Robotouille (Gonzalez-Pumariega et al., 2025) adds timing constraints. Its cooking tasks run with real delays and interruptions, so an agent must interleave subtasks instead of running them in sequence, and the cost of failing to do so is steep, with ReAct on GPT-4o falling from 47% on synchronous tasks to 11% on asynchronous ones.

From feasibility to solution quality. In every benchmark so far, the verdict is binary: a plan is feasible or it is not. This group turns to algo-

rithmic engineering, where the question is how good a solution is. ALE-Bench (Imajuku et al., 2026) grades each attempt with a score the agent improves over a long session, drawing its problems from AtCoder Heuristic Contests. Frontier models can match strong human contestants on a single problem, but they fall behind on the two things the format rewards most: consistency across problems and steady improvement over time. Across these benchmarks, a single success rate reveals little about why agents fail. The same failure modes appear across domains: each constraint is satisfied on its own but the plan violates them together, intermediate results get lost, a plan that looks plausible fails the checker, and an early mistake spreads until it can no longer be fixed. The benchmarks that catch these failures most reliably are the ones with automatic verifiers, such as TravelPlanner's constraint checks, DeepPlanning's sandboxes, and ALE-Bench's scorer. The model-level methods and harness-level mechanisms we review next can be judged against the same kind of automatic check.

4 Model-Level Determinants: Reducing Per-Step Error

Long-horizon failure is not a single catastrophic event but a compounding process: each step carries an independent probability of error, and these probabilities multiply across the execution sequence. Even a per-step success rate of 95%, applied over 100 actions, yields an end-to-end success probability below 1%. Section 5 examines how external execution infrastructure can compensate for model errors; this section asks what capabilities, internalized through inference-time computation or parameter updates, reduce the per-step error probability in the first place. We organize these model-level determinants along three axes: deliberate reasoning and search at inference time (§4.1), supervised internalization of tool use (§4.2), and reinforcement learning over full interaction trajectories (§4.3). Table 3 summarizes the representative methods.

4.1 Midtraining from Reasoning and Planning Traces

The earliest model-level interventions for reducing planning errors operate at inference time, requiring no parameter changes. **Chain-of-Thought (CoT) prompting** (Wei et al., 2022) elicits explicit intermediate reasoning steps before an agent commits to an action, reducing the frequency of locally incoherent decisions on arithmetic, commonsense, and symbolic tasks. While CoT externalizes single-path reasoning, it does not recover from dead ends. Tree of Thoughts (ToT) (Yao et al., 2023a) organizes intermediate states into a search tree, enabling backtracking and deliberate evaluation of alternative reasoning branches via breadth-first or depth-first search. ToT demonstrates that allocating more inference-time compute to structured search can substitute for weaker per-step reasoning.

Both CoT and ToT operate over reasoning traces disconnected from external environments. Language Agent Tree Search (LATS) (Zhou et al., 2023) bridges this gap by integrating Monte Carlo Tree Search (MCTS) with the ReAct (Yao et al., 2023c) loop, interleaving reasoning, grounded action execution, and language-based reflection within a unified tree search framework. LATS introduces environment feedback as a signal for tree-node evaluation, enabling recovery from execution errors rather than just reasoning errors.

Inference-time search methods are powerful, but their main model-level value lies in the traces they can provide for training. Plan-and-Act (Erdogan et al., 2025) further casts planning as a midtraining data problem. Rather than only separating a Planner from an Executor, it constructs grounded planning supervision from successful environment trajectories: a teacher model abstracts high-level plan steps from executed action traces and aligns them with the low-level actions that realize each step. This query–plan and plan–action data is then used to train models to produce executable decompositions instead of ungrounded plans. Empirically, high-quality dynamic planning substantially improves even a base Executor, raising WebArena-Lite success from 9.85% to 44.24%, and the full system reaches 57.58%.

The progression from CoT to LATS to Plan-and-Act reveals a consistent pattern: each method addresses the limitations of the previous one by introducing additional structure, including tree search, role specialization, and training. However, each also introduces new costs: compute overhead for search, sensitivity to evaluator quality, and the challenge of keeping plans synchronized with dynamic environments.

4.2 Tool-Use Learning

In grounded agent settings, planning errors and tool-use errors are distinct failure modes. An agent may reason correctly about what to do yet still se-

Regime	Representative methods	Long-horizon role	Primary limitation
Inference-time reasoning	CoT, ToT, LATS (Wei et al., 2022; Yao et al., 2023a; Zhou et al., 2023)	Adds intermediate reasoning, backtracking, and environment-grounded tree search before committing to actions.	High inference cost; dependent on evaluator quality.
Explicit planning	Plan-and-Act (Erdogan et al., 2025)	Separates high-level planning from environment-specific execution, reducing plan-generation bottlenecks.	Plans must remain synchronized with dynamic environments.
Tool-use learning	Toolformer, Gorilla (Schick et al., 2023; Patil et al., 2024)	Internalizes when to call tools and how to ground API invocation in documentation.	Static API distributions; weak task-level optimization.
Trajectory-level RL	LOOP (Chen et al., 2025b)	Optimizes agents directly in stateful interactive environments.	Coarse trajectory-level rewards and environment dependence.
Step-level credit	GiGPO, SALT, HCAPO, StepPO (Feng et al., 2026a; Li et al., 2026a; Tan et al., 2026; Wang et al., 2026b)	Assigns advantages to steps or states rather than uniformly to whole trajectories.	Often evaluated on moderate horizons; may require repeated states or multiple rollouts.
Stability and horizon scaling	SORL, KLong, IterResearch (Li et al., 2025a; Liu et al., 2026b; Chen et al., 2025a)	Stabilizes off-policy updates and scales training or inference to longer trajectories.	Expensive curricula, domain specificity, and difficult model-harness attribution.

Table 3: Representative model-level mechanisms for reducing per-step error. The regimes move from inference-time structure to tool-use internalization and long-horizon policy optimization.

lect the wrong API, generate invalid arguments, or misinterpret a tool's return value. The methods in §4.1 generally assume tool invocation is straightforward; this subsection asks what it takes to internalize reliable tool use through training.

Toolformer (Schick et al., 2023) pioneers self-supervised tool-use acquisition: the model generates its own training data by inserting candidate API calls into existing text, filtering those that reduce perplexity on subsequent tokens, and fine-tuning on the resulting corpus. This approach allows a model to learn *when* to call a tool without human annotation of tool-use opportunities. However, Toolformer's self-supervised signal is tied to next-token prediction and does not optimize for task-level outcomes across multi-step interactions.

As APIs proliferate, retrieval becomes essential.Gorilla (Patil et al., 2024) fine-tunes a model specifically for API invocation accuracy over a catalog of 16,000+ real-world APIs. By combining retrieval-augmented generation with instruction fine-tuning, Gorilla substantially reduces hallucination of invalid function signatures and achieves stronger performance than GPT-4 on API invocation benchmarks when documentation is retrieved rather than memorized.

Together, Toolformer and Gorilla establish two complementary principles: learning *when* to invoke tools as a temporal trigger and *which* tool to invoke accurately as API grounding. Both remain limited by their static training distributions: neither adapts dynamically when APIs change or deprecate. This limitation motivates environment-embedded RL approaches in §4.3.

4.3 Long-Horizon Agentic Reinforcement Learning

Supervised learning of reasoning and tool use reduces local step errors, but it does not directly optimize for long-horizon task completion. Distributional gaps between training trajectories and real deployment, compounding errors across steps, and the absence of dense per-step supervision collectively motivate reinforcement learning over full agent trajectories. Agentic RL presents four distinct challenges: sparse and delayed outcome rewards, credit assignment, training stability under off-policy updates, and context scaling when trajectories exceed context-window limits.

Trajectory-level training. LOOP (Chen et al., 2025b) establishes that RL training in stateful, multi-domain environments is feasible for LLM agents. Training interactive digital agents (IDAs) to use APIs of persistent application environments as a partially observable MDP, LOOP derives a memory-efficient PPO variant that maintains a single copy of the LLM without a value network. A

32B agent trained with LOOP on AppWorld surpasses the much larger OpenAI o1 by 9 percentage points, and analysis reveals that RL causes the agent to learn substantively useful behaviors: consulting API documentation, avoiding unwarranted assumptions, and recovering from execution failures.

Step-level credit assignment. A central difficulty in applying group-based RL, such as GRPO, to agent tasks is that sparse terminal rewards are assigned uniformly across all trajectory steps. Beneficial and detrimental actions receive identical reward signals when they co-occur in the same trajectory, inducing gradient conflicts during policy optimization.

GiGPO (Feng et al., 2026a) addresses this with a two-level group structure: at the episode level, macro advantages are computed from groups of complete trajectories sharing the same task; at the step level, an anchor-state grouping mechanism identifies repeated environment states across different rollouts and groups actions arising from the same state, enabling step-level micro-advantage estimation. GiGPO achieves greater than 12% improvement over GRPO on ALFWorld and greater than 9% on WebShop while maintaining identical GPU memory overhead and rollout cost.

SALT (Li et al., 2026a) constructs a trajectory graph from multiple rollouts of the same prompt, using cross-trajectory structural information to quantify the relative quality of each step and assign advantages accordingly. SALT is designed as a plug-and-play enhancement to existing group-based RL algorithms: it integrates with GRPO and RLOO without modifying the rollout procedure, introducing negligible computational overhead. Experiments on WebShop, ALFWorld, and AppWorld confirm consistent performance gains across model sizes.

HCAPO (Tan et al., 2026) takes a generative approach to credit: it uses the LLM itself as a post-hoc critic, injecting the known successful outcome into the prompt to simulate a hindsight distribution and refine step-level Q-value estimates through generative verification. A multi-scale advantage mechanism additionally corrects misaligned value baselines at critical decision states. StepPO (Wang et al., 2026b) approaches the same granularity mismatch from a different angle: rather than refining rewards, it reformulates the agentic MDP at the step level, treating interaction steps rather than tokens as the basic trajectory units for both modeling and policy optimization.

Stability and context scaling. Off-policy training exacerbates instability in long-horizon settings: token-level importance-sampling weights diverge as policy distributions drift across gradient updates, producing high-variance gradients and performance collapse. SORL (Li et al., 2025a) identifies two compounding sources of instability—token-level granularity mismatch and off-policy variance—and addresses them with turn-level importance sampling and clipping-triggered normalization. By computing importance weights at the turn boundary rather than the token level, SORL aligns credit correction with the natural interaction granularity of multi-turn agents, yielding consistently stable training curves and preventing the collapse observed in standard PPO and GRPO.

For tasks measured in hours of human effort, trajectories can span hundreds of turns and hundreds of thousands of tokens, far beyond standard context windows. KLong (Liu et al., 2026b) addresses this with a two-stage training recipe. First, trajectory-splitting SFT divides expert trajectories, distilled from Claude Sonnet with thinking, into overlapping sub-trajectories that preserve early context, progressively truncate later content, and maintain continuity across splits. Second, progressive RL schedules training across multiple stages with incrementally extended timeouts ($2 \rightarrow 4 \rightarrow 6$ hours), allowing the agent to gradually adapt to tasks with increasingly delayed reward structures. KLong at 106B parameters outperforms Kimi K2 Thinking (1T parameters) by 11.28% on PaperBench, demonstrating strong generalization to SWE-bench Verified and MLE-bench.

A complementary strategy addresses context growth at inference time. Mono-contextual agents append all retrieved information and reasoning steps to a single expanding context, leading to what IterResearch (Chen et al., 2025a) terms *context suffocation*: as context grows, earlier relevant information becomes harder to attend to, and noise accumulates. IterResearch reformulates long-horizon research as a Markov Decision Process in which the agent maintains an evolving workspace report rather than a raw context log, periodically synthesizing and reconstructing state. This Markovian reconstruction decouples exploration depth from context length, enabling scaling to 2048+ tool calls while holding context to approximately 40K tokens. Performance improves dramatically with interaction count, from 3.5% to 42.5%, a scaling law not

observed in mono-contextual baselines. IterResearch further develops Efficiency-Aware Policy Optimization (EAPO), which applies geometric reward discounting to incentivize efficient exploration and uses adaptive downsampling for stable distributed training.

5 Harness-Level Determinants: Extending the Reliable Horizon

Harness-level determinants concern the execution infrastructure that turns a language model into a stateful, tool-using, and evaluable agent. Model-level methods lower per-step error, but long-horizon execution fails through *accumulation*: even a high per-step success rate decays as a task lengthens—empirically close to a constant per-step hazard, giving each agent a characteristic success "half-life" (Ord, 2025). The *reliable* horizon is thus governed less by single-step competence than by whether the surrounding system contains and recovers from the errors that inevitably occur. Recent work formalizes this system as a "harness"—an externalization layer of execution loop, context manager, state store, and verification interface (Meng et al., 2026; Zhou et al., 2026a)—and shows it can push reliability far beyond a bare model: decomposition with per-step error correction has executed tasks of over a million steps with zero final errors (Meyerson et al., 2025). We organize the harness along three long-horizon-specific axes: grounding the execution loop in feedback, so per-step errors are caught and corrected before they cascade (§5.1); managing and persisting goal-relevant state—from the live context window to cross-episode memory and reusable skills—as the trajectory outgrows the window and spans episodes (§5.2); and sustaining *verified* autonomy by shifting outcome verification from the human to the agent (§5.3).

5.1 Execution Loops and Feedback

The foundational harness pattern is an execution loop that constructs the current state, elicits reasoning or an action, executes it, observes feedback, and conditions the next step on the result—turning one-shot generation into situated, feedback-driven execution. Within a *context-contained* episode, where the task state, recent observations, and stopping condition still fit the model's context, the loop improves reliability simply by making intermediate state visible and turning failures into signals. ReAct interleaves reasoning traces with environment-facing actions (Yao et al., 2023c), and closed-loop natural-language feedback lifts long-horizon embodied instruction-following without additional training (Huang et al., 2023). Feedback-driven refinement makes correction explicit: Reflexion converts outcome signals into verbal reflections (Shinn et al., 2023), Self-Refine iterates generate–feedback–refine (Madaan et al., 2023), Chain-of-Verification and Self-Debug check drafts against verification questions or execution results (Dhuliawala et al., 2024; Chen et al., 2024), and CRITIC grounds critique in external tools (Gou et al., 2024). Adaptive replanning and plan–execute decoupling keep the loop valid and efficient as horizons grow (Sun et al., 2023; Xu et al., 2023), while search-structured variants explore and backtrack over alternative paths when local decisions are ambiguous (Yao et al., 2023b; Zhou et al., 2024a). **??** organizes these in-context loops together with the state-management mechanisms taken up in §5.2.

These methods share one assumption—that the evidence needed to evaluate and repair an action stays visible in the current state—and one limit: intrinsic self-correction is unreliable when the model cannot tell it has erred. A critical survey finds that self-correction from prompted-model feedback alone rarely helps and works mainly when *reliable external feedback* is available (Kamoi et al., 2024). The practical response is to make the corrective signal external and trainable: learned critics catch more bugs than unaided human reviewers (McAleese et al., 2024), and generative process reward models verify each intermediate step rather than only the final answer (Khalifa et al., 2026). This shift—from internal-and-late toward external-and-early correction—is the first-order lever for long horizons, and it carries directly into the verification machinery of §5.3.

5.2 Carrying State: Context Management, Memory, and Skills

The feedback loop of §5.1 assumes that the evidence needed to evaluate and choose the next action stays inside the context window. Long-horizon tasks break this assumption: tool outputs, observations, code diffs, logs, and partial plans accumulate until the relevant state exceeds what the window can hold, while earlier decisions stay causally relevant long after they scroll out of view. The trajectory itself then becomes an object of harness control—selecting, compressing, externalizing, and recovering state under a finite context bud-

get (Wang et al., 2026d). Four mechanism families manage this state *within* the active task (lower half of **??**): *observation selection and pruning* trim high-volume tool outputs, logs, DOMs, or code before they overwhelm the next call (Wang et al., 2026e; Kovács, 2026; Shi et al., 2025; Kang et al., 2025; Lindenbauer et al., 2025; Lee et al., 2025); *trajectory summarization and folding* turn action–observation histories into compact continuation states (Wu et al., 2025; Ye et al., 2025; Sun et al., 2025; Xiao et al., 2026); *in-context memory construction and recovery* preserve facts, constraints, and dependencies that resurface many steps later within the same run (Zhou et al., 2025b; Wu et al., 2026a; Yang et al., 2026a); and *evidence and plan externalization* keep inspectable artifacts, citations, tests, and evolving plans outside the prompt (Li et al., 2025b; Hu et al., 2025a; Wu et al., 2026b). How state is selected, compressed, and recovered is itself a determinant of the reliable horizon: when the harness keeps the wrong evidence, compresses away an exact constraint, or fails to restore an earlier decision, later calls can be locally reasonable yet globally invalid.

These mechanisms keep a *single* trajectory usable; memory proper persists goal-relevant state *across* steps, episodes, and sessions, and skill libraries turn past success into reusable behavior, so that long or repeated tasks do not re-derive what has already been established. Following the working/episodic/semantic/procedural taxonomy of recent memory frameworks and surveys (Sumers et al., 2024; Hu et al., 2025b), *working and factual* memory retains and pages facts beyond the active window: Generative Agents couples a recency/importance/relevance memory stream with periodic reflection (Park et al., 2023), MemGPT treats the context window as virtual memory with OS-style paging (Packer et al., 2023), HippoRAG indexes experience into a knowledge graph for multi-hop recall (Gutiérrez et al., 2024), and Mem0 extracts and consolidates salient facts for production use, reporting on LOCOMO a 26% relative gain over a proprietary memory baseline at roughly 91% lower p95 latency and over 90% lower token cost (Chhikara et al., 2025). *Experiential* memory stores reusable episodes: ExpeL distills natural-language insights from past trajectories without weight updates (Zhao et al., 2024), A-MEM maintains an evolving Zettelkasten of dynamically linked notes (Xu et al., 2026d), and ACE curates accumulated experience as an evolving playbook (Zhang et al., 2026b).

Procedural memory and skill libraries store *how* to act: Agent Workflow Memory induces reusable workflows from prior trajectories (Wang et al., 2025f), Memp studies the build–retrieve–update lifecycle of procedural memory (Fang et al., 2026), and Voyager and JARVIS-1 accumulate executable, verified skills as composable building blocks for open-ended tasks (Wang et al., 2024a, 2025g). Across all three functions, externalized state survives context limits and goal drift and amortizes re-derivation over a long trajectory; its binding constraint is that memory can mislead as easily as it helps—a recent study of agent memory finds that no single architecture dominates across workloads, with effectiveness depending on matching memory structure to the task's bottleneck (Zhou et al., 2026c)—so stale or degraded state is itself a primary long-horizon failure mode.

5.3 Looping Engineering: Verified Autonomous Execution

Feedback and state keep a loop coherent; running it *long and unattended* additionally requires deciding, without a human, when output is good enough and when to stop. This is the premise of *looping engineering*: the agent finds work, executes, verifies its own outcome, and decides whether to continue, retry, escalate, or halt—moving the human from "in the loop" (reviewing each result) to "on the loop" (maintaining the harness) (Meng et al., 2026). Verification is the crux, and the hard part. As models strengthen, reliable verification becomes harder than generation and no fixed reward survives a continually improving policy, so the verifier must co-evolve with the agent (Wang et al., 2026a); moreover, purely automatic test-based gates degrade as tasks lengthen, leaving room for reward hacking on systems-level work (Zhao et al., 2026a). A loop also cannot safely be its own judge—letting the generating model gate itself tends to ratify its own mistakes—so verification is best delegated to an independent checker.

Agent-side verification and control. Several mechanisms supply this. Agent-as-a-Judge evaluates an agent's full trajectory with another agent rather than scoring only the endpoint (Zhuge et al., 2025); self-verifying coders interleave generation with self-generated tests (Jin et al., 2026); and process reward models score intermediate steps so the loop can prune or accept them at inference time (Choudhury, 2025). Knowing when to *stop* mat-

ters as much as what to do next: explicit early-exit policies break repetitive loops and hand off rather than churn (Lu et al., 2025), adversarial analysis shows that corrupting progress signals can trap agents in unbounded loops (Xu et al., 2026c), and scheduler-style control flow replaces the implicit loop with bounded, inspectable execution (Wei, 2026; Sypherd and Belle, 2024). When confidence is low the agent should escalate rather than proceed; studies of clarification timing find that frontier agents rarely ask in the right window (Gulati et al., 2026), and human-on-the-loop systems operationalize where to place such gates (Mozannar et al., 2025).

Evidence, limits, and failure. These ingredients let agents sustain coherent work over long durations: a self-reflective coding agent that verifies its own progress from an independent context has run unattended for tens of hours (Orimo et al., 2025). The same evidence cautions against equating autonomy with capability. A controlled scaling study finds that adding agents or coordination yields diminishing or negative returns once a single-agent baseline is strong and lacks centralized verification, with errors propagating through unchecked architectures (Kim et al., 2026); correspondingly, leaner designs—a fixed pipeline, or a production harness that is mostly a simple while-loop wrapped in operational infrastructure—often match elaborate ones at lower cost (Xia et al., 2025; Liu et al., 2026a; Wang et al., 2025b). Where loops still fail, empirical taxonomies trace the cause to non-termination, looping, and missing verification (Cemri et al., 2025; Zhu et al., 2025), motivating intervention-driven recovery that localizes the implicated step and reruns to confirm a repair (Ma et al., 2026). Table 4 summarizes these mechanisms.

Synthesis. Across the three axes, the reliable horizon is a property of the model–harness *pair*: the same model reaches very different horizons depending on the feedback that catches its errors, the state it can carry, and the verification that lets it run unattended. This is why separating model gains from harness gains is an open evaluation problem , and why we treat useful horizon as a systems property of the coupled model, harness, environment, and verifier.

Mechanism	Representative works
Why verification is hard	Verification Horizon, SpecBench (Wang et al., 2026a; Zhao et al., 2026a)
Agent-side verifiers	Agent-as-a-Judge, ReVeal, process reward models (Zhuge et al., 2025; Jin et al., 2026; Choudhury, 2025)
Termination & runaway control	early-exit, LoopTrap, structured control flow (Lu et al., 2025; Xu et al., 2026c; Wei, 2026)
Human on the loop / escalation	clarification timing, Magentic-UI (Gulati et al., 2026; Mozannar et al., 2025)
Long-autonomy evidence & failure	PARC, scaling caveat, failure taxonomies (Orimo et al., 2025; Kim et al., 2026; Cemri et al., 2025)

Table 4: Looping engineering: mechanisms for verified autonomous execution—moving outcome verification from the human to the agent so the loop can run long and unattended.

6 Challenges and Frontiers

The preceding sections show that long-horizon agents fail not because one component is missing, but because planning, grounding, memory, tools, verification, cost, and oversight interact across extended trajectories. This section distills the main open problems around the reliable execution horizon: coupled skill failures, memory degradation, error cascades and recovery, attribution, cost, contamination, and human oversight.

6.1 Planning, Grounding, and Tool-Use Failures

Planning, grounding, and tool use are often evaluated as separable capabilities, but long-horizon execution makes them tightly coupled. A correct high-level plan can fail if an agent grounds a browser element, file path, API field, or code location incorrectly; a correctly perceived state can still fail if the agent selects an invalid tool call or violates a domain policy; and a syntactically valid tool call can still be harmful if it advances the wrong subgoal (Deng et al., 2023; Zhou et al., 2024b; Patil et al., 2024; **?**). This coupling explains why benchmarks based on live or simulated environments often reveal failures that are invisible in static reasoning tasks (Xie et al., 2024b; Trivedi et al., 2024a).

A major frontier is therefore to evaluate failure composition rather than isolated skill accuracy. Web and GUI agents must connect language goals

to visual or structural observations over changing pages, software agents must map natural-language requirements to cross-file edits and executable tests, and tool agents must satisfy both user intent and hidden state constraints in databases or APIs (Koh et al., 2024; Jimenez et al., 2024; Wang et al., 2024b; **?**). In all cases, a local error can change the future state space: clicking the wrong control, modifying the wrong file, or updating the wrong database row creates new observations that may appear internally consistent while moving the trajectory away from the true goal (Wang et al., 2026c).

Progress requires diagnostic protocols that identify not only whether an agent failed, but which coupling failed first and how the failure propagated. Trajectory-level benchmarks such as AgentBoard and HORIZON move in this direction by emphasizing intermediate states and failure attribution, while stable tool-evaluation infrastructure reduces confounds caused by changing external APIs or nondeterministic graders (Ma et al., 2024; Wang et al., 2026c; Guo et al., 2024). Future evaluations should annotate whether failures originate in decomposition, observation grounding, tool selection, argument construction, policy compliance, state tracking, or verification, because each source implies a different model or harness intervention (Rabanser et al., 2026).

6.2 Memory Degradation and Goal Drift

Long-horizon agents must decide what to remember, what to summarize, what to retrieve, what to discard, and what to distrust. Memory is therefore not merely a larger context window: it is a state-management mechanism that must preserve user constraints, intermediate artifacts, tool outputs, environment observations, and unresolved subgoals while preventing stale or false information from contaminating future decisions (Packer et al., 2023; Gutiérrez et al., 2024; Hu et al., 2025b; Du, 2026). This distinction matters because a memory system that stores more text can still shorten the reliable horizon if it retrieves outdated state or compresses away task-critical constraints (Mei et al., 2025; Wu et al., 2026b).

Goal drift arises when an agent's active objective gradually diverges from the user's original intent or from the current environment state. It can occur through lossy summaries, irrelevant retrieval, repeated self-generated assumptions, failed attempts copied into scratchpads, or over-optimization of local subgoals such as passing a test while violating a broader specification (Shinn et al., 2023; Shi et al., 2025; Yan et al., 2026). In coding and workflow tasks, this drift is especially dangerous because intermediate artifacts become part of the world: a wrong edit, invalid cached result, or obsolete plan may be repeatedly reinforced by later observations (Jimenez et al., 2024; Wang et al., 2026c).

The research frontier is to evaluate memory by *state fidelity* and *action usefulness*, not by storage capacity alone. Useful memory should preserve task-critical invariants, identify the freshness and provenance of stored facts, support contradiction detection, and expose uncertainty when retrieved information may no longer match the environment (Xu et al., 2026d; Du, 2026; Mei et al., 2025). Harnesses can help by separating durable user constraints from temporary observations, attaching timestamps and tool provenance to memory entries, maintaining checkpoints, and forcing revalidation before irreversible actions (Qiao et al., 2023; Wang et al., 2025e; Mozannar et al., 2025).

6.3 Error Cascades, Self-Correction, and Recovery

Long-horizon language agents do not make decisions from a stateless context. Each step is conditioned on an accumulated trajectory of prior actions, observations, tool responses, intermediate plans, and, in some systems, persistent memory. As a result, an early mistake may not simply cause a local failure; it can be incorporated into the agent state and subsequently shape later reasoning and action selection. This phenomenon gives rise to **error cascades**, where downstream decisions may appear coherent with the recorded trajectory while still being causally downstream of an earlier fault. Recent diagnostic studies make this issue explicit: TOOLSCAN (Kokane et al.) moves beyond final success rate by characterizing fine-grained tool-use errors, such as invalid formats, hallucinated functions, incorrect arguments, repeated calls, and insufficient API calls, while AgentErrorTaxonomy (Zhu et al., 2025) attributes failures to memory, reflection, planning, action, and system-level modules and highlights error propagation as a central bottleneck in agent reliability.

A common mitigation strategy is to introduce iterative critique and revision. Self-Refine (Madaan et al., 2023) turns generation into a feedback and refinement loop in which the same model critiques and revises its own output, whereas Reflexion (Shinn et al., 2023) stores verbal reflections from

failed trials as episodic memory to guide subsequent attempts. These methods show that feedback-conditioned revision can improve agent behavior in many settings. However, their success should not be conflated with reliable intrinsic self-correction. In reasoning tasks, models often fail to judge the correctness of their own outputs without additional evidence, and self-correction can even reduce performance (Huang et al., 2024). This suggests that recovery is less a matter of simply generating another response than of identifying which part of the previous trajectory should be revised, ignored, or discarded.

External feedback provides a more grounded basis for such diagnosis. Tool outputs, execution traces, verifier judgments, simulator observations, and environment errors can expose discrepancies between an agent's intended plan and the actual state of the world. CRITIC (Gou et al., 2024) illustrates this principle by verifying model outputs through external tools and using the resulting critiques to guide revision. In agentic planning, Hell or High Water (Wang et al., 2025a) isolates a complementary setting: agents encounter external tool failures while the task remains solvable through alternative function compositions, thereby testing whether they can abandon a failed plan and construct a backup strategy. PALADIN (Vuddanti et al., 2025) further treats recovery as a learnable execution-level behavior: it injects tool failures into ToolBench-style trajectories, annotates recovery actions, fine-tunes agents on failure-rich rollouts, and retrieves taxonomy-aligned recovery exemplars at inference time to guide retry, tool substitution, replanning, or graceful termination. These studies indicate that recovery requires not only retrying, but also interpreting feedback, localizing the failure source, and adapting the subsequent action space.

In summary, this line of work reframes agent robustness as a state-management problem. A recovery protocol must specify the failure type, the observability of the failure signal, whether the task remains feasible, what alternative actions are available, and how much of the accumulated trajectory should continue to be trusted. Open challenges remain in distinguishing recoverable external failures from invalid tasks, detecting silent semantic failures that produce plausible but wrong observations, and deciding when persistence should give way to safe abstention. Error cascades, self-correction, and recovery should therefore be studied jointly, as mechanisms for controlling how unreliable trajectory state is diagnosed, revised, and propagated.

6.4 Research Frontiers: Attribution, Cost, Contamination, and Human Oversight

Method-level progress alone does not determine whether language agents are reliable. Modern agents are increasingly deployed as systems: their behavior is shaped not only by the base model, but also by context assembly, memory retrieval, tool interfaces, search and verification policies, execution controllers, and human intervention. Recent surveys on context engineering, self-evolving agents, and agent memory make this systems view explicit, treating context, memory, tools, and adaptation as first-class determinants of agent behavior rather than auxiliary prompting choices (Mei et al., 2025; Gao et al.; Hu et al., 2025b; Du, 2026). This shift raises a methodological question that current benchmarks only partially address: when an agent succeeds, what exactly should the success be attributed to, under what resource budget, with what contamination risk, and under what autonomy constraints?

Model–harness attribution. A central challenge is to disentangle model capability from harness effects. Improvements on agent benchmarks may reflect a stronger base model, but they may also arise from better context construction, memory retrieval, tool schemas, search depth, verifier design, retry policies, or interface constraints. Final task success is therefore insufficient evidence of model-level progress. More rigorous evaluation should report both model-only and system-level performance, and should include controlled ablations that vary one component while holding the others fixed. This requires, for example, freezing the harness while changing the model, freezing the model while changing memory or tool policies, and reporting cross-combinations of models and scaffolds. Trajectory-centric benchmarks such as AgentBoard move in this direction by evaluating agents through multi-step action–observation processes rather than single-shot outputs (Ma et al., 2024). However, the field still lacks standardized factorial protocols for attributing gains across model, memory, tool, verifier, and execution-policy components.

Cost-normalized evaluation. Agent reliability should also be compared under explicit resource budgets. Higher success rates can often be obtained by increasing prompt length, sampling more candidates, deepening search, adding reflection rounds,

calling more tools, using stronger verifiers, or coordinating larger agent teams. Without cost normalization, benchmark comparisons may conflate capability with expenditure. Recent work on scaling agent systems shows that coordination benefits are highly task-dependent: multi-agent systems can help on decomposable tasks but can also incur coordination overhead, saturation effects, and topology-dependent error amplification on sequential or tool-heavy tasks (Kim et al., 2026). Evaluation should therefore report success jointly with token usage, wall-clock latency, number of model calls, tool-call counts, verifier cost, memory growth, and human-intervention cost. In many settings, Pareto frontiers over accuracy, cost, and latency are more informative than a single aggregate success score.

Benchmark contamination. Contamination is particularly difficult to control in agentic evaluation. Leakage may occur not only through final answers, but also through task templates, tool schemas, website states, demonstrations, grading scripts, public solution traces, recovery exemplars, or benchmark-specific prompting conventions. Thus, strong performance on a long-horizon benchmark may reflect familiarity with the evaluation environment rather than robust generalization. General studies of benchmark data contamination distinguish multiple levels of exposure, from semantic and information-level leakage to direct data and label leakage (Xu et al., 2024). Dynamic benchmarks provide complementary mitigation strategies: LatestEval (Li et al., 2024) constructs reading-comprehension tests from recent texts, LiveCodeBench (Jain et al., 2025) uses time-stamped programming problems and evaluates models on post-cutoff data, and LiveBench (White et al., 2024) combines frequently updated questions with objective ground-truth scoring. For agents, these ideas need to be extended beyond fresh questions to regenerated environments, rotating tool schemas, hidden tool states, private task distributions, and trace-level leakage audits.

Adaptive autonomy and human oversight. A further frontier is determining when agents should act autonomously and when they should defer to human judgment. Oversight should not be reduced to a binary choice between full automation and manual control. Depending on risk and reversibility, an agent may need to request clarification, seek approval before irreversible actions, checkpoint its state, roll back a previous step, abstain under uncertainty, or escalate to a human operator. This issue is acute in tool-using and state-changing environments, where errors can affect external systems rather than merely produce incorrect text. Safety benchmarks such as AgentDojo (Debenedetti et al., 2024) and AgentSafetyBench (Zhang et al., 2024) show that tool-integrated agents introduce behavior-level risks, including unsafe tool invocation, prompt-injection vulnerability, excessive trust in tool outputs, and insufficient risk awareness. Governance-oriented work further emphasizes constrained action spaces, approval gates, audit trails, sandboxing, rollback mechanisms, and accountability for deployed agentic systems (Shavit et al., 2023). Future protocols should treat human intervention as a scarce but necessary resource, reporting escalation precision, approval burden, intervention latency, rollback success, reversibility, and post-hoc trace interpretability.

In summary, these frontiers shift the unit of analysis from isolated task success to the quality of evidence supporting a reliability claim. A mature evaluation protocol should disclose the base model, harness configuration, context policy, memory policy, tool budget, verifier budget, contamination controls, autonomy policy, and human-intervention interface. Without such reporting, progress remains difficult to attribute, expensive to reproduce, and unsafe to extrapolate to open-ended deployment.

7 Conclusion: Reliable Horizon Is a Systems Property

LLM agents are increasingly expected to solve tasks that require extended interaction with tools, software environments, web interfaces, databases, and other mutable external states. Long-horizon LLM agents mark a shift from isolated model responses to sustained, stateful, and goal-directed execution. They must plan across multiple stages, preserve task context, coordinate tool use, verify intermediate outcomes, and recover from failures as errors accumulate over dependent trajectories. This survey examines long-horizon LLM agents through four perspectives: horizon characterization, model capability, harness design, and evaluation methodology, while introducing reliable execution horizon as a unifying lens for organizing existing work, exposing key limitations, and motivating future research on dependable long-horizon agency.

References

Petr Anokhin, Roman Khalikov, Stefan Rebrikov, Viktor Volkov, Artyom Sorokin, and Vincent Bissonnette. 2025. Herobench: A benchmark for long-horizon planning and structured reasoning in virtual worlds. *arXiv preprint arXiv:2508.12782*.

Sagnik Anupam, Davis Brown, Shuo Li, Eric Wong, Hamed Hassani, and Osbert Bastani. 2025. Browserarena: Evaluating llm agents on real-world web navigation tasks. *arXiv preprint arXiv:2510.02418*.

Ibragim Badertdinov, Maksim Nekrashevich, Anton Shevtsov, and Alexander Golubev. 2026. Swerebench v2: Language-agnostic swe task collection at scale. *arXiv preprint arXiv:2602.23866*.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.

Rogerio Bonatti, Dan Zhao, Francesco Bonacci, Dillon Dupont, Sara Abdali, Yinheng Li, Yadong Lu, Justin Wagle, Kazuhito Koishida, Arthur Bucker, and 1 others. 2025. Windows agent arena: Evaluating multimodal os agents at scale. In *International Conference on Machine Learning*, pages 4874–4910. PMLR.

Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei A Zaharia, Joseph Gonzalez, and Ion Stoica. 2025. Why do multi-agent llm systems fail? In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc.

Guoxin Chen, Zile Qiao, Xuanzhong Chen, Donglei Yu, Haotian Xu, Wayne Xin Zhao, Ruihua Song, Wenbiao Yin, Huifeng Yin, Liwen Zhang, and 1 others. 2025a. Iterresearch: Rethinking long-horizon agents with interaction scaling. *arXiv preprint arXiv:2511.07327*.

Jialong Chen, Xander Xu, Hu Wei, Chuan Chen, and Bing Zhao. 2026. Swe-ci: Evaluating agent capabilities in maintaining codebases via continuous integration. *Preprint*, arXiv:2603.03823.

Kevin Chen, Marco Cusumano-Towner, Brody Huval, Aleksei Petrenko, Jackson Hamburger, Vladlen Koltun, and Philipp Krähenbühl. 2025b. Reinforcement learning for long-horizon interactive llm agents. *arXiv preprint arXiv:2502.01600*.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching large language models to self-debug. In *International Conference on Learning Representations*, volume 2024, pages 8746–8825.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.

Sanjiban Choudhury. 2025. Process reward models for llm agents: Practical framework and directions. *Preprint*, arXiv:2502.10325.

Evan Chu, Rajan Agarwal, Abishek Thangamuthu, Brendan Graham, Justus Mattern, Freeman Jiang, Paul Cento, Swarnim Jain, Mersad Abbasi, Mohammad Hossein Rezaei, George Wang, Alex Zhang, Simon Guo, Karina Nguyen, Danna Liu, Arash Bidgoli, Aditya Dalmia, Apoorv Dankar, Ashrut Vaddela, and 12 others. 2026. Frontierswe. *Proximal Blog*. Https://frontierswe.com/blog.

Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in Neural Information Processing Systems*, 37:82895–82920.

Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Vijay Bharadwaj, Jeff Holm, Raja Aluri, Chen Bo Calvin Zhang, and 3 others. 2025. Swebench pro: Can ai agents solve long-horizon software engineering tasks? *Preprint*, arXiv:2509.16941.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems*, 36:28091–28114.

Rishi Desai, Jesse Hu, Joan Cabezas, Neel Harsola, Pratyush Shukla, Roey Ben Chaim, Adnan El Assadi, Omkaar Mukund Kamath, Fenil Faldu, Prannay Hebbar, Jiankai Sun, Yiyuan Li, Pramod Srinivasan, Ishan Gupta, Christopher Settles, Daniel Wang, Derek Chen, Pranav Raja, Albert Liu, and 7 others. 2026. Swe-marathon: Can agents autonomously complete ultra-long-horizon software work? *Preprint*, arXiv:2606.07682.

Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. 2024. Chain-of-verification reduces hallucination in large language models. In *Findings of the association for computational linguistics: ACL 2024*, pages 3563–3578.

Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, and 30 others. 2026. Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents. *Preprint*, arXiv:2512.12730.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David

Vazquez, and 1 others. 2024. Workarena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*.

Mingxuan Du, Benfeng Xu, Chiwei Zhu, Xiaorui Wang, and Zhendong Mao. 2025. Deepresearch bench: A comprehensive benchmark for deep research agents. *arXiv preprint arXiv:2506.11763*.

Pengfei Du. 2026. Memory for autonomous llm agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*.

Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. 2025. Plan-and-act: Improving planning of agents for long-horizon tasks. *arXiv preprint arXiv:2503.09572*.

Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. 2026. Memp: Exploring agent procedural memory. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 17490–17502, San Diego, California, United States. Association for Computational Linguistics.

Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. 2026a. Group-in-group policy optimization for llm agent training. *Advances in Neural Information Processing Systems*, 38:46375–46408.

Yukang Feng, Jianwen Sun, Zelai Yang, Jiaxin Ai, Chuanhao Li, Zizhen Li, Fanrui Zhang, Kang He, Rui Ma, Jifan Lin, Jie Sun, Yang Xiao, Sizhuo Zhou, Wenxiao Wu, Yiming Liu, Pengfei Liu, Yu Qiao, Shenglin Zhang, and Kaipeng Zhang. 2026b. Longcli-bench: A preliminary benchmark and study for long-horizon agentic programming in command-line interfaces. *Preprint*, arXiv:2602.14337.

Huan-ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Qihan Ren, and 1 others. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*.

Gonzalo Gonzalez-Pumariega, Leong Yean, Neha Sunkara, and Sanjiban Choudhury. 2025. Robotouille: An asynchronous planning benchmark for llm agents. In *International Conference on Learning Representations*, volume 2025, pages 83757–83792.

Boyu Gou, Zanming Huang, Yuting Ning, Yu Gu, Michael Lin, Weijian Qi, Andrei Kopanev, Botao Yu, Bernal Jiménez Gutiérrez, Yiheng Shu, and 1 others. 2025. Mind2web 2: Evaluating agentic search with agent-as-a-judge, 2025. *URL https://arxiv.org/abs/2506.21506*.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Nan Duan, Weizhu Chen, and 1 others. 2024. Critic: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, volume 2024, pages 57734–57811.

Hao Guan, Lingyue Fu, Shao Zhang, Yaoming Zhu, Kangning Zhang, Lin Qiu, Xunliang Cai, Xuezhi Cao, Weiwen Liu, Weinan Zhang, and Yong Yu. 2026. Swe-cycle: Benchmarking code agents across the complete issue resolution cycle. *Preprint*, arXiv:2605.13139.

Anmol Gulati, Hariom Gupta, Elias Lumer, Sahil Sen, and Vamse Kumar Subbiah. 2026. Ask early, ask late, ask right: When does clarification timing matter for long-horizon agents? *Preprint*, arXiv:2605.07937.

Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Peng, Zhiyuan Liu, Maosong Sun, and Yang Liu Yang. 2024. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*.

Bernal J Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in neural information processing systems*, 37:59532–59569.

Songhao Han, Boxiang Qiu, Yue Liao, Siyuan Huang, Chen Gao, Shuicheng Yan, and Si Liu. 2026. Robocerebra: A large-scale benchmark for long-horizon robotic manipulation evaluation. *Advances in Neural Information Processing Systems*, 38.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6864–6890.

William Hill, Ireton Liu, Anita De Mello Koch, Damion Harvey, Nishanth Kumar, George Konidaris, and Steven James. 2023. Mineplanner: A benchmark for long-horizon planning in large minecraft worlds. *arXiv preprint arXiv:2312.12891*.

Jiseung Hong, Benjamin G. Ascoli, and Jinho D. Choi. 2026. Recube: Evaluating repository-level context utilization in code generation. *Preprint*, arXiv:2603.25770.

Chen Hu, Haikuo Du, Heng Wang, Lin Lin, Mingrui Chen, Peng Liu, Ruihang Miao, Tianchi Yue, Wang You, Wei Ji, and 1 others. 2025a. Step-deepresearch technical report. *arXiv preprint arXiv:2512.20491*.

Ruida Hu, Xinchen Wang, Chao Peng, Cuiyun Gao, and David Lo. 2026. Evaluating llm-based 0-to-1 software generation in end-to-end cli tool scenarios. *Preprint*, arXiv:2604.06742.

Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, and 1 others. 2025b.

Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *International conference on learning representations*, volume 2024, pages 32808–32824.

Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Tomas Jackson, Noah Brown, Linda Luu, Sergey Levine, Karol Hausman, and brian ichter. 2023. Inner monologue: Embodied reasoning through planning with language models. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 1769–1782. PMLR.

Wenqi Huang and Peter Jiang. 2026. Deepswe v1.1: a cleaner, more reproducible benchmark for frontier coding agents.

Yuki Imajuku, Kohki Horie, Yoichi Iwata, Kensho Aoki, Naohiro Takahashi, and Takuya Akiba. 2026. Ale-bench: A benchmark for long-horizon objective-driven algorithm engineering. *Advances in Neural Information Processing Systems*, 38.

Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. 2025. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, pages 58791–58831.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157.

Yiyang Jin, Kunzhao Xu, Hang Li, Xueting Han, Yanmin Zhou, Cheng Li, and Jing Bai. 2026. Reveal: Self-evolving code agents via reliable self-verification. In *The Fourteenth International Conference on Learning Representations*.

Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. 2024. When can llms actually correct their own mistakes? a critical survey of self-correction of llms. *Transactions of the Association for Computational Linguistics*, 12:1417–1440.

Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. 2025. Acon: Optimizing context compression for long-horizon llm agents. *arXiv preprint arXiv:2510.00615*.

Raghav Kapoor, Yash Parag Butala, Melisa Russak, Jing Yu Koh, Kiran Kamble, Waseem AlShikh, and Ruslan Salakhutdinov. 2024. Omniact: A dataset and benchmark for enabling multimodal generalist autonomous agents for desktop and web. In *European Conference on Computer Vision*, pages 161–178. Springer.

Sayash Kapoor, Benedikt Stroebl, Peter Kirgis, Nitya Nadgir, Zachary S Siegel, Boyi Wei, Tianci Xue, Ziru Chen, Felix Chen, Saiteja Utpala, and 1 others. 2025. Holistic agent leaderboard: The missing infrastructure for ai agent evaluation. *arXiv preprint arXiv:2510.11977*.

Muhammad Khalifa, Rishabh Agarwal, Lajanugen Logeswaran, Jaekyeom Kim, Hao Peng, Moontae Lee, Honglak Lee, and Lu Wang. 2026. Process reward models that think. *Transactions on Machine Learning Research*. J2C Certification.

Aaditya Khanal, Yangyang Tao, and Junxiu Zhou. 2026. Beyond pass@ 1: A reliability science framework for long-horizon llm agents. *arXiv preprint arXiv:2603.29231*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations*.

Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Yun Liu, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, and Xin Liu. 2026. Towards a science of scaling agent systems. *Preprint*, arXiv:2512.08296.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. 2024. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905.

Shirley Kokane, Ming Zhu, Tulika Manoj Awalgaonkar, Jianguo Zhang, Akshara Prabhakar, Thai Quoc Hoang, Zuxin Liu, Rithesh RN, Liangwei Yang, Weiran Yao, and 1 others. Toolscan: A benchmark for characterizing errors in tool-use llms. In *ICLR 2025 Workshop on Building Trust in Language Models and Applications*.

Ádám Kovács. 2026. Squeez: Task-conditioned tool-output pruning for coding agents. *arXiv preprint arXiv:2604.04979*.

Thomas Kwa, Ben West, Joel Becker, Amy Deng, Katharyn Garcia, Max Hasin, Sami Jawhar, Megan Kinniment, Nate Rush, Sydney Von Arx, and 1 others. 2026. Measuring ai ability to complete long software

tasks. *Advances in Neural Information Processing Systems*, 38:92213–92266.

Man Ho Lam, Chaozheng Wang, Hange Liu, Jingyu Xiao, Haau sing Li, Jen tse Huang, Terry Yue Zhuo, and Michael R. Lyu. 2026. Swe-chain: Benchmarking coding agents on chained release-level package upgrades. *Preprint*, arXiv:2605.14415.

Tue Le, Minh V. T. Thai, Dung Nguyen Manh, Huy Phan Nhat, and Nghi D. Q. Bui. 2026. Swe-evo: Benchmarking coding agents in long-horizon software evolution scenarios. *Preprint*, arXiv:2512.18470.

Dongjun Lee, Juyong Lee, Kyuyoung Kim, Jihoon Tack, Jinwoo Shin, Yee Whye Teh, and Kimin Lee. 2025. Learning to contextualize web pages for enhanced decision making by LLM agents. In *The Thirteenth International Conference on Learning Representations*.

Juyong Lee, Taywon Min, Minyong An, Dongyoon Hahm, Haeone Lee, Changyeon Kim, and Kimin Lee. 2024. Benchmarking mobile device control agents across diverse configurations. *arXiv preprint arXiv:2404.16660*.

Ido Levy, Ben Wiesel, Sami Marreed, Alon Oved, Avi Yaeli, Nir Mashkif, and Segev Shlomov. 2024. St-webagentbench: A benchmark for evaluating safety and trustworthiness in web agents. *arXiv preprint arXiv:2410.06703*.

Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Michael Lingelbach, Jiankai Sun, and 1 others. 2023. Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning*, pages 80–93. PMLR.

Chenliang Li, Adel Elmahdy, Alex Boyd, Zhongruo Wang, Siliang Zeng, Alfredo Garcia, Parminder Bhatia, Taha Kass-Hout, Cao Xiao, and Mingyi Hong. 2025a. Stabilizing off-policy training for long-horizon llm agent via turn-level importance sampling and clipping-triggered normalization. *arXiv preprint arXiv:2511.20718*.

Jiazheng Li, Yawei Wang, Qiaojing Yan, Yijun Tian, Zhichao Xu, Huan Song, Panpan Xu, and Lin Lee Cheong. 2026a. Salt: Step-level advantage assignment for long-horizon agents via trajectory graph. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 4709–4725.

Wanli Li, Bowen Zhou, Yunyao Yu, Zhou Xu, Yifan Yang, Dongsheng Li, and Caihua Shan. 2026b. Weavebench: A long-horizon, real-world benchmark for computer-use agents with hybrid interfaces. *arXiv preprint arXiv:2606.09426*.

Yucheng Li, Frank Guerin, and Chenghua Lin. 2024. Latesteval: Addressing data contamination in language model evaluation through dynamic and time-sensitive test construction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 18600–18607.

Zijian Li, Xin Guan, Bo Zhang, Shen Huang, Houquan Zhou, Shaopeng Lai, Ming Yan, Yong Jiang, Pengjun Xie, Fei Huang, and 1 others. 2025b. Webweaver: Structuring web-scale evidence with dynamic outlines for open-ended deep research. *arXiv preprint arXiv:2509.13312*.

Tobias Lindenbauer, Igor Slinko, Ludwig Felder, Egor Bogomolov, and Yaroslav Zharov. 2025. The complexity trap: Simple observation masking is as efficient as llm summarization for agent context management. *arXiv preprint arXiv:2508.21433*.

Jiacheng Liu, Xiaohan Zhao, Xinyi Shang, and Zhiqiang Shen. 2026a. Dive into claude code: The design space of today's and future ai agent systems. *Preprint*, arXiv:2604.14228.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, and 1 others. 2024. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046.

Yue Liu, Yingwei Ma, Yibo Miao, Yanhao Li, Yuchong Xie, Xinlong Yang, Zhiyuan Hu, Flood Sung, Jiaheng Zhang, and Bryan Hooi. 2026b. Klong: Training llm agent for extremely long-horizon tasks. *arXiv preprint arXiv:2602.17547*.

Qingyu Lu, Liang Ding, Siyi Cao, Xuebo Liu, Kanjian Zhang, Jinxia Zhang, and Dacheng Tao. 2025. Runaway is ashamed, but helpful: On the early-exit behavior of large language model-based agents in embodied environments. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 24014–24027, Suzhou, China. Association for Computational Linguistics.

Xing Han Lù, Zdenek Kasner, and Siva Reddy. Weblinx: Real-world website navigation with multi-turn dialogue, 2024. *URL https://arxiv. org/abs/2402.05930*, 3(8).

Haotian Luo, Huaisong Zhang, Xuelin Zhang, Haoyu Wang, Zeyu Qin, Wenjie Lu, Guozheng Ma, Haiying He, Yingsha Xie, Qiyang Zhou, and 1 others. 2025. Ultrahorizon: Benchmarking agent capabilities in ultra long-horizon scenarios. *arXiv preprint arXiv:2509.21766*.

Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. Agentboard: An analytical evaluation board of multi-turn llm agents. *Advances in neural information processing systems*, 37:74325–74362.

Ming Ma, Jue Zhang, Fangkai Yang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2026. Dover: Intervention-driven auto debugging for llm multi-agent systems.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594.

Nat McAleese, Rai Michael Pokorny, Juan Felipe Ceron Uribe, Evgenia Nitishinskaya, Maja Trebacz, and Jan Leike. 2024. Llm critics help catch llm bugs. *arXiv preprint arXiv:2407.00215*.

Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, and 1 others. 2025. A survey of context engineering for large language models. *arXiv preprint arXiv:2507.13334*.

Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. 2026. Agent harness for large language model agents: A survey.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, and 66 others. 2026. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *Preprint*, arXiv:2601.11868.

Elliot Meyerson, Giuseppe Paolo, Roberto Dailey, Hormoz Shahrzad, Olivier Francon, Conor F Hayes, Xin Qiu, Babak Hodjat, and Risto Miikkulainen. 2025. Solving a million-step llm task with zero errors. *arXiv preprint arXiv:2511.09030*.

Mahmoud Mohammadi, Yipeng Li, Jane Lo, and Wendy Yip. 2025. Evaluation and benchmarking of llm agents: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 6129–6139.

Hussein Mozannar, Gagan Bansal, Cheng Tan, Adam Fourney, Victor Dibia, Jingya Chen, Jack Gerrits, Tyler Payne, Matheus Kunzler Maldaner, Madeleine Grunde-McLaughlin, Eric Zhu, Griffin Bassman, Jacob Alber, Peter Chang, Ricky Loynd, Friederike Niedtner, Ece Kamar, Maya Murad, Rafah Hosn, and Saleema Amershi. 2025. Magentic-ui: Towards human-in-the-loop agentic systems. *Preprint*, arXiv:2507.22358.

Rabeya Khatun Muna, Md Nakhla Rafi, Tse-Hsun, and Chen. 2026. Ci-repair-bench: A repository-aware benchmark for automated patch validation via ci workflows. *Preprint*, arXiv:2604.27148.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, and 1 others. 2022. Webgpt: Browser-assisted question-answering with human feedback, 2022. *URL https://arxiv. org/abs/2112.09332*, 35.

OpenAI. 2024. Introducing swe-bench verified.

Toby Ord. 2025. Is there a half-life for the success rates of ai agents? *arXiv preprint arXiv:2505.05115*.

Yuki Orimo, Iori Kurata, Hodaka Mori, Ryuhei Okuno, Ryohto Sawada, and Daisuke Okanohara. 2025. Parc: An autonomous self-reflective coding agent for robust execution of long-horizon tasks. *Preprint*, arXiv:2512.03549.

Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. Memgpt: towards llms as operating systems.

Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, and 1 others. Webcanvas: Benchmarking web agents in online environments, 2024. *URL https://arxiv. org/abs/2406.12373*.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22.

Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. 2025. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565.

Adamenko Pavel, Ivanov Mikhail, Aidar Valeev, Rodion Levichev, Pavel Zadorozhny, Ivan Lopatin, Dmitrii Babaev, Alena Fenogenova, and Valentin Malykh. 2025. Swe-mera: A dynamic benchmark for agenticly evaluating large language models on software engineering tasks. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 440–452.

Zhiyuan Peng, Xin Yin, Pu Zhao, Fangkai Yang, Lu Wang, Ran Jia, Xu Chen, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2026. Repogenesis: Benchmarking end-to-end microservice generation from readme to repository. *Preprint*, arXiv:2601.13943.

Bo Qiao, Liqun Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Zhang, Fangkai Yang, Hang Dong, Jue Zhang, Lu Wang, and 1 others. 2023. Taskweaver: A code-first agent framework. *arXiv preprint arXiv:2311.17541*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2024. Toolllm: Facilitating large language models to master 16000+ real-world

apis. In *International Conference on Learning Representations*, volume 2024, pages 9695–9717.

Stephan Rabanser, Sayash Kapoor, Peter Kirgis, Kangheng Liu, Saiteja Utpala, and Arvind Narayanan. 2026. Towards a science of ai agent reliability. *arXiv preprint arXiv:2602.16666*.

Mohit Raghavendra, Soham Dan, Miguel Romero Calvo, Yannis Yiming He, Johannes Baptist Mols, Gautam Anand, Cole McCollum, Edgar Arakelyan, Vijay Bharadwaj, Andrew Park, Jeff Da, MohammadHossein Rezaei, Bing Liu, Brad Kenstler, and Yunzhong He. 2026. Swe atlas: Benchmarking coding agents beyond issue resolution. *Preprint*, arXiv:2605.08366.

Guruprasad Viswanathan Ramesh, Asmit Nayak, Basieem Siddique, and Kassem Fawaz. 2026. Websp-eval: Evaluating web agents on website security and privacy tasks. *arXiv preprint arXiv:2604.06367*.

Ben Rank, Hardik Bhatnagar, Ameya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and Maksym Andriushchenko. 2026. Posttrainbench: Can llm agents automate llm post-training? *Preprint*, arXiv:2603.08640.

Chris Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyo Campbell-Ajala, and 1 others. 2025. Androidworld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations*, volume 2025, pages 406–441.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551.

Ye Shang, Quanjun Zhang, Haichuan Hu, Chunrong Fang, Liang Xiao, and Zhenyu Chen. 2026. Breaking, stale, or missing? benchmarking coding agents on project-level test evolution. *Preprint*, arXiv:2605.06125.

Yonadav Shavit, Sandhini Agarwal, Miles Brundage, Steven Adler, Cullen O'Keefe, Rosie Campbell, Teddy Lee, Pamela Mishkin, Tyna Eloundou, Alan Hickey, and 1 others. 2023. Practices for governing agentic ai systems. *Research Paper, OpenAI*.

Tianlin Shi, Andrej Karpathy, Linxi Fan, Jonathan Hernandez, and Percy Liang. 2017. World of bits: An open-domain platform for web-based agents. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3135–3144. PMLR.

Yuling Shi, Yichun Qian, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2025. Longcodezip: Compress long context for code language models. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 141–153.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652.

Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. 2020. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10740–10749.

Akshit Sinha, Arvindh Arun, Shashwat Goel, Steffen Staab, and Jonas Geiping. 2025. The illusion of diminishing returns: Measuring long horizon execution in llms. *arXiv preprint arXiv:2509.09677*.

{Theodore R.} Sumers, Shunyu Yao, Karthik Narasimhan, and {Thomas L.} Griffiths. 2024. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. Publisher Copyright: © 2024, Transactions on Machine Learning Research. All rights reserved.

Haotian Sun, Yuchen Zhuang, Lingkai Kong, Bo Dai, and Chao Zhang. 2023. Adaplanner: Adaptive planning from feedback with language models. *Advances in neural information processing systems*, 36:58202–58245.

Weiwei Sun, Miao Lu, Zhan Ling, Kang Liu, Xuesong Yao, Yiming Yang, and Jiecao Chen. 2025. Scaling long-horizon llm agent via context-folding. *arXiv preprint arXiv:2510.11967*.

Zihao Sun and Ling Chen. 2025. Webarxiv: Evaluating multimodal agents on time-invariant arxiv tasks. *arXiv preprint arXiv:2507.00938*.

Chris Sypherd and Vaishak Belle. 2024. Practical considerations for agentic llm systems. *Preprint*, arXiv:2412.04093.

Hui-Ze Tan, Xiao-Wen Yang, Hao Chen, Jie-Jing Shao, Yi Wen, Yuteng Shen, Weihong Luo, Xiku Du, Lan-Zhe Guo, and Yu-Feng Li. 2026. Hindsight credit assignment for long-horizon llm agents. *arXiv preprint arXiv:2603.08754*.

Natalia Tarasova, Enrique Balp-Straffon, Aleksei Iancheruk, Yevhenii Sielskyi, Nikita Kozodoi, Liam H. Byrne, Jack Butler, Dayuan Jiang, Marcin Czelej, Andrew Ang, Yash Shah, Roi Blanco, and Sergei Ivanov. 2026. Swe-infrabench: Evaluating language models on cloud infrastructure code. *Preprint*, arXiv:2606.05249.

George Thomas, Alex J Chan, Jikun Kang, Wenqi Wu, Filippos Christianos, Fraser Greenlee, Andy Toulis, and Marvin Purtorab. 2025. Webgames: Challenging general-purpose web-browsing ai agents. *arXiv preprint arXiv:2502.18356*.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024a. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076.

Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024b. Appworld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076.

Ada Defne Tur, Nicholas Meade, Xing Han Lù, Alejandra Zambrano, Arkil Patel, Esin Durmus, Spandana Gella, Karolina Stańczak, and Siva Reddy. 2025. Safearena: Evaluating the safety of autonomous web agents. *arXiv preprint arXiv:2503.04957*.

Sri Vatsa Vuddanti, Aarav Shah, Satwik Kumar Chittiprolu, Tony Song, Sunishchal Dev, Kevin Zhu, and Maheep Chaudhary. 2025. Paladin: Self-correcting language model agents to cure tool-failure cases. *arXiv preprint arXiv:2509.25238*.

Andrew Wang, Sophia Hager, Adi Asija, Daniel Khashabi, and Nicholas Andrews. 2025a. Hell or high water: Evaluating agentic recovery from external failures. In *Second Conference on Language Modeling*.

Binghai Wang, Chenlong Zhang, Dayiheng Liu, Jiajun Zhang, Jiawei Chen, Mingze Li, Mouxiang Chen, Rongyao Fang, Siyuan Zhang, Xuwu Wang, Yuheng Jing, Zeyao Ma, and Zeyu Cui. 2026a. The verification horizon: No silver bullet for coding agent rewards. *Preprint*, arXiv:2606.26300.

Daoyu Wang, Qingchuan Li, Mingyue Cheng, Jie Ouyang, Shuo Yu, Qi Liu, and Enhong Chen. 2026b. Steppo: Step-aligned policy optimization for agentic reinforcement learning. *arXiv preprint arXiv:2604.18401*.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024a. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*.

Haoran Wang, Zhenyu Hou, Yao Wei, Jie Tang, and Yuxiao Dong. 2025b. SWE-dev: Building software engineering agents with training and inference scaling. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 3742–3761, Vienna, Austria. Association for Computational Linguistics.

Maria Wang, Srinivas Sunkara, Gilles Baechler, Jason Lin, Yun Zhu, Fedir Zubach, Lei Shu, and Jindong Chen. Webquest: A benchmark for multimodal qa on web page sequences. corr, abs/2409.13711, 2024. doi: 10.48550. *arXiv preprint ARXIV.2409.13711*.

Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. 2025c. Odysseybench: Evaluating llm agents on long-horizon complex office application workflows. *arXiv preprint arXiv:2508.09124*.

Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. 2025d. Odysseybench: Evaluating llm agents on long-horizon complex office application workflows. *arXiv preprint arXiv:2508.09124*.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024b. Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, and 1 others. 2025e. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919.

Xingyao Wang, Zihan Wang, Jiateng Liu, Yangyi Chen, Lifan Yuan, Hao Peng, and Heng Ji. 2024c. Mint: Evaluating llms in multi-turn interaction with tools and language feedback. In *International Conference on Learning Representations*, volume 2024, pages 32593–32627.

Xinyu Jessica Wang, Haoyue Bai, Yiyou Sun, Haorui Wang, Shuibai Zhang, Wenjie Hu, Mya Schroder, Bilge Mutlu, Dawn Song, and Robert D Nowak. 2026c. The long-horizon task mirage? diagnosing where and why agentic systems break. *arXiv preprint arXiv:2604.11978*.

Yifei Wang, Ziteng Wang, Yuling Shi, Silin Chen, Xinrui Wang, Yueqi Wang, Beijun Shen, Linjing Li, Xiaodong Gu, Julian McAuley, and 1 others. 2026d. Context compression for llm agents: A survey of methods, failure modes, and evaluation.

Yuhang Wang, Yuling Shi, Mo Yang, Rongrui Zhang, Shilin He, Heng Lian, Yuting Chen, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026e. Swe-pruner: Self-adaptive context pruning for coding agents. *arXiv preprint arXiv:2601.16746*.

Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025f. Agent workflow memory.

Zihao Wang, Shaofei Cai, Anji Liu, Yonggang Jin, Jinbing Hou, Bowei Zhang, Haowei Lin, Zhaofeng He, Zilong Zheng, Yaodong Yang, Xiaojian Ma, and Yitao Liang. 2025g. JARVIS-1: Open-World Multi-Task Agents With Memory-Augmented Multimodal Language Models . *IEEE Transactions on Pattern Analysis & Machine Intelligence*, 47(03):1894–1907.

Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. 2024d. Officebench: Benchmarking language agents across multiple applications for office automation. *arXiv preprint arXiv:2407.19056*.

Hu Wei. 2026. From agent loops to structured graphs:a scheduler-theoretic framework for llm agent execution. *Preprint*, arXiv:2604.11378.

Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. 2025. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Siddartha Naidu, and 1 others. 2024. Livebench: A challenging, contamination-free llm benchmark. *arXiv preprint arXiv:2406.19314*, 4:2.

Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, and 1 others. 2024. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*.

Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, and 1 others. 2025. Resum: Unlocking long-horizon search intelligence via context summarization. *arXiv preprint arXiv:2509.13313*.

Yating Wu, Yuhao Zhang, Sayan Ghosh, Sourya Basu, Anoop Deoras, Jun Huan, and Gaurav Gupta. 2026a. Contextweaver: Selective and dependency-structured memory construction for llm agents. *arXiv preprint arXiv:2604.23069*.

Yong Wu, YanZhao Zheng, TianZe Xu, ZhenTao Zhang, YuanQiang Yu, JiHuai Zhu, Chao Ma, BinBin Lin, BaoHua Dong, HangCheng Zhu, and 1 others. 2026b. Contextbudget: Budget-aware context management for long-horizon search agents. *arXiv preprint arXiv:2604.01664*.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Agentless: Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE).

Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2026. Reducing cost of llm agents with trajectory reduction. *Proceedings of the ACM on Software Engineering*, 3(FSE):1241–1263.

Jian Xie, Kai Zhang, Jiangjie Chen, Tinghui Zhu, Renze Lou, Yuandong Tian, Yanghua Xiao, and Yu Su. 2024a. Travelplanner: A benchmark for real-world planning with language agents. *arXiv preprint arXiv:2402.01622*.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, and 1 others. 2024b. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094.

Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. 2023. Rewoo: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*.

Cheng Xu, Shuhao Guan, Derek Greene, M Kechadi, and 1 others. 2024. Benchmark data contamination of large language models: A survey. *arXiv preprint arXiv:2406.04244*.

Frank Fangzheng Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, and 1 others. 2026a. Theagentcompany: benchmarking llm agents on consequential real world tasks. *Advances in Neural Information Processing Systems*, 38.

Frank Fangzheng Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, and 1 others. 2026b. Theagentcompany: benchmarking llm agents on consequential real world tasks. *Advances in Neural Information Processing Systems*, 38.

Huiyu Xu, Zhibo Wang, Wenhui Zhang, Ziqi Zhu, Yaopeng Wang, Kui Ren, and Chun Chen. 2026c. Looptrap: Termination poisoning attacks on llm agents. *Preprint*, arXiv:2605.05846.

Kevin Xu, Yeganeh Kordi, Tanay Nayak, Adi Asija, Yizhong Wang, Kate Sanders, Adam Byerly, Jingyu Zhang, Benjamin Van Durme, and Daniel Khashabi. Tur [k] ingbench: A challenge benchmark for web agents, 2025. *URL https://arxiv.org/abs/2403.11905*.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2026d. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems*, 38:17577–17604.

Xinbo Xu, Ruihan Yang, Haiyang Shen, Wendong Xu, Bofei Gao, Ruoyu Wu, Kean Shi, Weichu Xie, Xuanzhong Chen, Ming Wu, Jason Zeng, Michael Heinrich, Elvis Zhang, Liang Chen, Kuan Li, and Baobao Chang. 2026e. Roadmapbench: Evaluating long-horizon agentic software development across version upgrades. *Preprint*, arXiv:2605.15846.

Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An illusion of progress? assessing the current state of web agents, 2025. *URL https://arxiv.org/abs/2504.01382*.

Lu Yan, Xuan Chen, and Xiangyu Zhang. 2026. When the specification emerges: Benchmarking faithfulness loss in long-horizon coding agents. *arXiv preprint arXiv:2603.17104*.

Chengyuan Yang, Zequn Sun, Wei Wei, and Wei Hu. 2026a. Beyond static summarization: Proactive memory extraction for llm agents. *arXiv preprint arXiv:2601.04463*.

John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin, Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. 2026b. Programbench: Can language models rebuild programs from scratch? *Preprint*, arXiv:2605.03546.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023b. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023c. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, and 1 others. 2025. Agentfold: Long-horizon web agents with proactive context management. *arXiv preprint arXiv:2510.24699*.

Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2026. A survey on evaluation of llm-based agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 26690–26714.

Zonghao Ying, Yangguang Shao, Jianle Gan, Gan Xu, Wenxin Zhang, Quanchen Zou, Junzheng Shi, Zhenfei Yin, Mingchuan Zhang, Aishan Liu, and 1 others. 2026. Securewebarena: A holistic security evaluation benchmark for lvlm-based web agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 11986–11998.

Ori Yoran, Samuel Joseph Amouyal, Chaitanya Malaviya, Ben Bogin, Ofir Press, and Jonathan Berant. 2024. Assistantbench: Can web agents solve realistic and time-consuming tasks? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 8938–8968.

Linghao Zhang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Chengxing Xie, Junhao Wang, Maoquan Wang, Yufan Huang, Shengyu Fu, and 1 others. 2026a. Swe-bench goes live! *Advances in Neural Information Processing Systems*, 38.

Linghao Zhang, Junhao Wang, Shilin He, Chaoyun Zhang, Yu Kang, Bowen Li, Jiaheng Wen, Chengxing Xie, Maoquan Wang, Yufan Huang, and 1 others. 2025a. Di-bench: Benchmarking large language models on dependency inference with testable repositories at scale. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 10134–10153.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. *Preprint*, arXiv:2510.04618.

Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and Xiaodong Gu. 2026c. Swe-explore: Benchmarking how coding agents explore repositories. *Preprint*, arXiv:2606.07297.

Yinger Zhang, Shutong Jiang, Renhao Li, Jianhong Tu, Yang Su, Lianghao Deng, Xudong Guo, Chenxu Lv, and Junyang Lin. 2026d. Deepplanning: Benchmarking long-horizon agentic planning with verifiable constraints. *arXiv preprint arXiv:2601.18137*.

Zehua Zhang, Ati Priya Bajaj, Divij Handa, Siyu Liu, Arvind S Raj, Hongkai Chen, Hulin Wang, Yibo Liu, Zion Leonahenahe Basque, Souradip Nath, Vishal Juneja, Nikhil Chapre, Yan Shoshitaishvili, Adam Doupé, Chitta Baral, and Ruoyu Wang. 2025b. Buildbench: Benchmarking llm agents on compiling real-world open-source software. *Preprint*, arXiv:2509.25248.

Zhexin Zhang, Shiyao Cui, Yida Lu, Jingzhuo Zhou, Junxiao Yang, Hongning Wang, and Minlie Huang. 2024. Agent-safetybench: Evaluating the safety of llm agents. *arXiv preprint arXiv:2412.14470*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642.

Bingchen Zhao, Dhruv Srikanth, Yuxiang Wu, and Zhengyao Jiang. 2026a. Specbench: Measuring reward hacking in long-horizon coding agents. *Preprint*, arXiv:2605.21384.

Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, and 1 others. 2026b. Ama-bench: Evaluating long-horizon memory for agentic applications. *arXiv preprint arXiv:2602.22769*.

Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024a. Gpt-4v (ision) is a generalist web agent, if grounded. *arXiv preprint arXiv:2401.01614*.

Huaixiu Steven Zheng, Swaroop Mishra, Hugh Zhang, Xinyun Chen, Minmin Chen, Azade Nova, Le Hou, Heng-Tze Cheng, Quoc V Le, Ed H Chi, and 1 others. 2024b. Natural plan: Benchmarking llms on natural language planning. *arXiv preprint arXiv:2406.04520*.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. 2023. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and {Yu Xiong} Wang. 2024a. Language agent tree search unifies reasoning, acting, and planning in language models. *Proceedings of Machine Learning Research*, 235:62138–62160. We thank Daniel Campos for useful feedback on earlier versions of this paper.This work was supported in part by NSF Grant 2106825, NIFA Award 2020-67021-32799, the Jump ARCHES endowment through the Health Care Engineering Systems Center at Illinois and the OSF Foundation, and the IBM-Illinois Discovery Accelerator Institute.This work used NVIDIA GPUs at NCSA Delta through allocations CIS220014, CIS230012, and CIS230218 from the ACCESS program.; 41st International Conference on Machine Learning, ICML 2024 ; Conference date: 21-07-2024 Through 27-07-2024.

Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, and 1 others. 2026a. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering. *arXiv preprint arXiv:2604.08224*.

Peilin Zhou, Bruce Leon, Xiang Ying, Can Zhang, Yifan Shao, Qichen Ye, Dading Chong, Zhiling Jin, Chenxuan Xie, Meng Cao, and 1 others. 2025a. Browsecomp-zh: Benchmarking web browsing ability of large language models in chinese. *arXiv preprint arXiv:2504.19314*.

Qixing Zhou, Jiacheng Zhang, Haiyang Wang, Rui Hao, Jiahe Wang, Minghao Han, Yuxue Yang, Shuzhe Wu, Feiyang Pan, Lue Fan, Dandan Tu, and Zhaoxiang Zhang. 2026b. Featurebench: Benchmarking agentic coding for complex feature development. *Preprint*, arXiv:2602.10975.

Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, and 1 others. 2024b. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, volume 2024, pages 15585–15606.

Wei Zhou, Xuanhe Zhou, Shaokun Han, Hongming Xu, Guoliang Li, Zhiyu Li, Feiyu Xiong, and Fan Wu. 2026c. Are we ready for an agent-native memory system? *Preprint*, arXiv:2606.24775.

Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. 2025b. Mem1: Learning to synergize memory and reasoning for efficient long-horizon agents. *arXiv preprint arXiv:2506.15841*.

Kunlun Zhu, Zijia Liu, Bingxuan Li, Muxin Tian, Yingxuan Yang, Jiaxun Zhang, Pengrui Han, Qipeng Xie, Fuyang Cui, Weijia Zhang, Xiaoteng Ma, Xiaodong Yu, Gowtham Ramesh, Jialian Wu, Zicheng Liu, Pan Lu, James Zou, and Jiaxuan You. 2025. Where llm agents fail and how they can learn from failures. *Preprint*, arXiv:2509.25370.

Mingchen Zhuge, Changsheng Zhao, Dylan R. Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, Yangyang Shi, Vikas Chandra, and Jürgen Schmidhuber. 2025. Agent-as-a-judge: Evaluate agents with agents. In *Forty-second International Conference on Machine Learning*.

A Example Appendix

This is an appendix.